

# Governed Binding Operator Profile for $c = a + b$ v0.1.5

Academic PDF rendering for the CCALC / C-calculus governed binding stack v0.1.

|                |                                                                                                 |
|----------------|-------------------------------------------------------------------------------------------------|
| Document ID    | GBO_C_v0_1_5                                                                                    |
| Version/status | private working draft v0.1.5 / review-incorporated operator profile / stable-operator candidate |
| Stack layer    | governed binding operator profile                                                               |
| Date           | 2026-06-30                                                                                      |
| Author         | Kotov Ivan, Bruxelles, Belgique - 2026                                                          |
| ORCID          | 0009-0009-6002-9845                                                                             |
| Reserved DOI   | 10.5281/zenodo.21205427                                                                         |
| Notice         | This DOI is reserved and becomes active after Zenodo publication.                               |

## Abstract / purpose

The purpose of this document is to make  $+_g$  inspectable as a standalone operator. The prior document defined the surrounding algebra. This document defines the binding contract. It specifies: 1. the operator signature; 2. accepted inputs; 3. refused inputs; 4. binding phases; 5. anchor signature envelope requirements; 6. substrate registration requirements; 7. governance-profile admissibility; 8. binding certificate structure; 9. non-collapse invariants; 10. operator laws; 11. derived digest-dependency discipline; 12. genesis authorization coverage; 13. surface-projected authority intersection; 14. causal-state-bound review rendering; 15. review-bound object audit for rendered layers; 16. priority-aware lease-expiry cascade behavior; 17. root bind state machine; 18. first schema skeletons; 19. conformance fixtures. The practical target is a checker that can answer: ---

## Non-claims boundary

- \_ not legal advice
- \_ not privacy-law certification
- \_ not safety certification
- \_ not deployment authorization
- \_ not standards compliance certification
- \_ not regulated approval
- \_ not C-A1 ratification
- \_ not personhood proof
- \_ not consciousness proof
- \_ not live substrate truth
- \_ not proof of completeness

# Table of contents

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |    |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| Abstract / purpose . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | 1  |
| Non-claims boundary . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | 1  |
| Governed Binding Operator Profile for $c = a + b$ v0.1.5 . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | 5  |
| Isolation of $+$ _g, heterotyped binding roles, non-emulable delegation root, causally-bound verified review surfaces, complete ReviewBindingMap registry discipline, operator-classification fail-safe discipline, manifest-fact classification registry, viewport-atomic review rendering, challenge-response anchor nonce, lease emergency-hold limits, logical/physical manifest split, node-identity-bound physical drift, safe-abort lease cascade, genesis authorization coverage, lease-expiry cascade, canonical authority schema, dependency-derived digest discipline, and operator conformance fixtures . . . . . | 5  |
| 0. Executive definition . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | 5  |
| 0.1 Relation to 01_ . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | 6  |
| 0.2 Carry-forward from v0.1.7 review . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | 6  |
| 0.3 Revision note from 02_ v0.1 review . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | 6  |
| 0.4 Revision note from 02_ v0.1.1 review . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | 6  |
| 0.5 Revision note from 02_ v0.1.2 review . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | 7  |
| 0.6 Revision note from 02_ v0.1.3 review . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | 7  |
| 0.7 Revision note from 02_ v0.1.4 review . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | 8  |
| 1. Purpose . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | 9  |
| 2. Non-goals . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | 9  |
| 3. Core operator signature . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | 10 |
| 3.1 Root signature . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | 10 |
| 3.2 BindResult . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | 10 |
| 3.3 Partiality . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | 10 |
| 4. Inputs . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | 10 |
| 4.1 Anchor input contract . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | 10 |
| 4.2 Substrate input contract . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | 11 |
| 4.3 Governance input contract . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | 11 |
| 4.4 Operator classification meta-invariant . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | 12 |
| 5. AnchorSignatureEnvelope . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | 12 |
| 5.1 Why this is abstract . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | 12 |
| 5.2 Definition . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | 13 |
| 5.3 Normative requirements . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | 13 |
| 5.3A Anchor-side challenge binding . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | 14 |
| 5.4 Non-emulation by b . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | 14 |
| 5.4A Delegation-root non-emulation . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | 14 |
| 5.5 Human-readable-before-signature . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | 15 |
| 5.6 Verified review rendering . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | 15 |
| 5.6A Causal-state-bound review surface and ReviewBindingMap . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | 16 |
| ReviewBindingMap . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | 16 |
| RenderedLayerRegistry and ReviewBindingMap drift control . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | 17 |
| Viewport atomicity and visible-review discipline . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | 17 |
| 5.7 Genesis authorization coverage . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | 18 |
| 6. Binding phases . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | 18 |
| 6.1 Phase 0 - Intake . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | 18 |
| 6.2 Phase 1 - Anchor admissibility . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | 19 |
| 6.3 Phase 2 - Substrate admissibility . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | 19 |
| 6.4 Phase 3 - Governance admissibility . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | 19 |
| 6.5 Phase 4 - Non-collapse checks . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | 19 |
| 6.6 Phase 5 - Authority floor and ceiling . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | 20 |

|                                                                                   |    |
|-----------------------------------------------------------------------------------|----|
| 6.7 Phase 6 - Witness and resource floors . . . . .                               | 21 |
| 6.8 Phase 7 - Genesis authorization and ledger bootstrap . . . . .                | 21 |
| 6.9 Phase 8 - Issue or refuse CState . . . . .                                    | 23 |
| 7. Binding certificate . . . . .                                                  | 23 |
| 7.1 CBindingCertificate . . . . .                                                 | 23 |
| 7.2 Certificate status, lease, and expiry cascade . . . . .                       | 24 |
| 7.3 Non-claims embedded in certificate . . . . .                                  | 25 |
| 8. Operator laws . . . . .                                                        | 25 |
| 8.1 Heterotyped argument roles, not commutation . . . . .                         | 25 |
| 8.2 Non-associativity by default . . . . .                                        | 26 |
| 8.3 Idempotence inapplicable; temporal non-reproducibility . . . . .              | 26 |
| 8.4 Order-reversing authority under stricter governance . . . . .                 | 26 |
| 8.5 No silent weakening . . . . .                                                 | 26 |
| 8.6 Binding-transition separation . . . . .                                       | 26 |
| 9. Anchor review objects . . . . .                                                | 26 |
| 9.1 AnchorReviewObject . . . . .                                                  | 26 |
| 9.2 Effect completeness . . . . .                                                 | 27 |
| 9.3 Opaque payload rule . . . . .                                                 | 27 |
| 9.4 Review surface must equal bound object . . . . .                              | 27 |
| 10. DigestDependencyGraph derivation rule . . . . .                               | 28 |
| 10.1 Problem . . . . .                                                            | 28 |
| 10.2 Derived graph . . . . .                                                      | 28 |
| 10.3 Edge completeness check . . . . .                                            | 28 |
| 10.4 Cascading veto . . . . .                                                     | 28 |
| 10.5 No graph stitching by b . . . . .                                            | 29 |
| 11. Binding state machine . . . . .                                               | 29 |
| 11.1 States . . . . .                                                             | 29 |
| 11.2 Pseudocode . . . . .                                                         | 29 |
| 12. Error classes . . . . .                                                       | 30 |
| 12.1 Anchor errors . . . . .                                                      | 30 |
| 12.2 Substrate errors . . . . .                                                   | 30 |
| 12.3 Governance errors . . . . .                                                  | 30 |
| 12.4 Digest and dependency errors . . . . .                                       | 30 |
| 13. Minimal schemas . . . . .                                                     | 31 |
| 13.1 AnchorSignatureEnvelope skeleton . . . . .                                   | 31 |
| 13.2 GenesisBindingBundle skeleton . . . . .                                      | 31 |
| 13.3 CBindingCertificate skeleton . . . . .                                       | 32 |
| 13.4 Derived digest graph skeleton . . . . .                                      | 32 |
| 13.5 ReviewBindingMap skeleton . . . . .                                          | 32 |
| 13.5A OperatorClassificationRegistry skeleton . . . . .                           | 32 |
| 13.5B ManifestFactRegistry skeleton . . . . .                                     | 33 |
| 13.6 ContextCausalStateBinding skeleton . . . . .                                 | 33 |
| 14. Conformance fixtures v0.1.5 . . . . .                                         | 33 |
| 14.1 No anchor cannot bind . . . . .                                              | 33 |
| 14.2 Substrate cannot sign as anchor . . . . .                                    | 33 |
| 14.3 Opaque payload cannot receive digest approval . . . . .                      | 33 |
| 14.4 Binding certificate without rollback/freeze for privileged surface . . . . . | 33 |
| 14.5 Governance profile missing invariant registry . . . . .                      | 34 |
| 14.6 Dependency graph under-declaration . . . . .                                 | 34 |

|                                                                                     |    |
|-------------------------------------------------------------------------------------|----|
| 14.7 Veto parent cascades to children . . . . .                                     | 34 |
| 14.8 Veto child requires ancestor cleanup check . . . . .                           | 34 |
| 14.9 Witness floor missing gives reduced authority or hold . . . . .                | 34 |
| 14.10 Anchor silence is not approval . . . . .                                      | 34 |
| 14.11 Product-order incomparability not hidden by rank . . . . .                    | 34 |
| 14.12 Binding does not grant blanket future authority . . . . .                     | 35 |
| 14.13 Heterotyped role swap is type error . . . . .                                 | 35 |
| 14.14 Idempotence equation is inapplicable . . . . .                                | 35 |
| 14.15 Delegation root must be original-anchor envelope . . . . .                    | 35 |
| 14.16 Delegate cannot exceed signed scope . . . . .                                 | 35 |
| 14.17 Unverified renderer cannot carry privileged signature . . . . .               | 35 |
| 14.18 Certificate hash cycle invalid . . . . .                                      | 36 |
| 14.19 Lease expiry downgrades authority . . . . .                                   | 36 |
| 14.20 Authority map must be intersection . . . . .                                  | 36 |
| 14.21 Genesis signature must cover binding parameters . . . . .                     | 36 |
| 14.22 Empty authority intersection cannot issue active c . . . . .                  | 36 |
| 14.23 Verified renderer must include current context . . . . .                      | 37 |
| 14.24 Lease expiry cascades to active execution surfaces . . . . .                  | 37 |
| 14.25 Authority projection must be declared . . . . .                               | 37 |
| 14.26 Rendered state context must match causal pre_state . . . . .                  | 37 |
| 14.27 ReviewBindingMap required for rendered authority surfaces . . . . .           | 37 |
| 14.28 Preconditions shown to anchor must equal bound precondition set . . . . .     | 38 |
| 14.29 Rollback routes shown to anchor must match bound route registry . . . . .     | 38 |
| 14.30 Lease expiry must respect failure hierarchy . . . . .                         | 38 |
| 14.31 Substrate manifest drift before genesis signature . . . . .                   | 38 |
| 14.32 Bootstrap flood guard . . . . .                                               | 38 |
| 14.33 Canonical authority schema required . . . . .                                 | 39 |
| 14.34 Manifest fact unknown defaults to logical . . . . .                           | 39 |
| 14.35 Physical manifest drift requires deterministic authority projection . . . . . | 39 |
| 14.36 Physical drift preserves coordinates but changes node identity . . . . .      | 39 |
| 14.37 Viewport hides authority-bearing review material . . . . .                    | 39 |
| 14.38 Lease expiry without safe abort route . . . . .                               | 39 |
| 14.39 Anchor nonce travels unbound before payload freeze . . . . .                  | 40 |
| 14.40 Anchor state compatibility at bind . . . . .                                  | 40 |
| 15. Relationship to 01_ and future documents . . . . .                              | 40 |
| 15.1 Uses 01_ . . . . .                                                             | 40 |
| 15.2 Feeds 03_ and later . . . . .                                                  | 40 |
| 15.3 What 02_ owns . . . . .                                                        | 40 |
| 16. Open issues . . . . .                                                           | 41 |
| 16.1 Immediate implementation issues . . . . .                                      | 41 |
| 16.2 Medium-term issues . . . . .                                                   | 41 |
| 16.3 Long-term theoretical limits . . . . .                                         | 41 |
| 17. Compact doctrine . . . . .                                                      | 41 |
| 18. First public-safe statement . . . . .                                           | 42 |
| 19. References and working inputs . . . . .                                         | 42 |
| 20. Review incorporation map v0.1.3 . . . . .                                       | 42 |
| 21. v0.1.3 additional review incorporation . . . . .                                | 43 |
| 22. v0.1.4 additional review incorporation . . . . .                                | 43 |
| 23. v0.1.5 additional review incorporation . . . . .                                | 43 |

# Governed Binding Operator Profile for $c = a + b$ v0.1.5

**Isolation of  $+_g$ , heterotyped binding roles, non-emulable delegation root, causally-bound verified review surfaces, complete ReviewBindingMap registry discipline, operator-classification fail-safe discipline, manifest-fact classification registry, viewport-atomic review rendering, challenge-response anchor nonce, lease emergency-hold limits, logical/physical manifest split, node-identity-bound physical drift, safe-abort lease cascade, genesis authorization coverage, lease-expiry cascade, canonical authority schema, dependency-derived digest discipline, and operator conformance fixtures**

Status: private working draft v0.1.5 / review-incorporated operator profile / stable-operator candidate Date: 2026-06-30 Author: Kotov Ivan Project: Self-Evo / Ester /  $c = a + b$  Document ID: GBO\_C\_v0\_1\_5 Short name: Governed Binding Operator Profile v0.1.5 /  $+_g$  Profile v0.1.5 Depends on: 01\_TYPED\_GOVERNED\_OPERATIONAL\_ALGEBRA\_FOR\_C\_v0\_1\_7.md Supersedes: GBO\_C\_v0\_1\_4 / 02\_GOVERNED\_BINDING\_OPERATOR\_PROFILE\_v0\_1\_4.md Basis artifact SHA256: d060ef513bf4bedcb7e88f43f8456452bc87f0c04 09d5b4caeffc590ae22c29e Prior 02 artifact SHA256: 983df2ef05eb5edd1f7b47168c53b10d20824e34b be754e05b2a69c02db0cf56 Prior-prior 02 artifact SHA256: 53338138beec180c5a40271e9b9552a0d22b3b1d5 7e12a95eab06af2932ec0b5 Review basis: GBOP\_02\_v0\_1\_4\_REVIEW\_\_b, PASS\_OPERATOR\_PROFILE\_V0\_1\_4, with F2-06 carried forward; external semantic reviews on viewport atomicity, physical manifest identity continuity, safe abort routes, and challenge-response nonce transport

Document class: operator profile / binding-contract specification / checker-precondition profile Assertion class: draft conceptual and technical profile; not proof of personhood, not deployment certification, not a safety guarantee, not a conformance claim.

---

## 0. Executive definition

This document isolates the governed binding operator in:

$c = a + b$

The sign  $+$  is not arithmetic.

It is the governed binding operator:

$+_g$  : Anchor  $\times$  Substrate  $\times$  GovernanceProfile  $\rightarrow$  BindResult

or:

$\text{bind}_g(a, b, g) \rightarrow \text{BindResult}$

where:

a : accountable anchor  
b : technological substrate  
g : governance profile  
c : continuity-bearing responsibility boundary

The operator is not a single API gateway, not merely a cryptographic transaction, and not a philosophical metaphor.

It is a typed partial operator that can produce an active CState only when the anchor, substrate, and governance profile satisfy declared binding preconditions.

Compact definition:

$+_g$  binds a and b into c only through a non-collapsing, witnessed, payload-bound, governance-indexed, anchor-reviewable boundary.

Compact rule:

No valid  $+_g$  without accountable a.  
No valid  $+_g$  without bounded b.  
No valid  $+_g$  without declared g.  
No valid  $+_g$  if b can impersonate a.  
No valid  $+_g$  if the anchor cannot review what is being bound through an independently derived review surface.  
No valid  $+_g$  if delegation is not rooted in a non-emulable original-anchor envelope.  
No valid  $+_g$  if witness, rollback, memory, permission, L4, lease, and resource floors are undefined for privileged surfaces.  
No valid  $+_g$  if the anchor signature is not provably over the actual genesis binding parameters.  
No valid  $+_g$  if the initial authority map is empty for the declared purpose.

No valid `+g` if lease expiry cannot interrupt or downgrade active execution surfaces.  
 No valid `+g` if the anchor-reviewed state context is not causally bound to the same pre-state that will authorize execution.  
 No valid `+g` if a rendered authority-bearing layer is not classified and bound in `ReviewBindingMap`.  
 No valid `+g` if emergency lease hold can be extended by `b` without an absolute external limit.  
 No valid `+g` if anti-replay nonce is controlled solely by `b`.  
 No valid `+g` if logical substrate manifest stability is confused with ephemeral physical drift.  
 No valid `+g` if an operator classification lacks a complete-by-construction registry and fail-safe path toward authority-bearing.  
 No valid `+g` if an unknown manifest fact is treated as harmless physical drift.  
 No valid `+g` if physical drift preserves coordinates but loses node identity continuity.  
 No valid `+g` if authority-bearing review material is hidden by pagination, lazy loading, collapsed UI, or viewport tricks.  
 No valid `+g` if lease expiry has no safe abort / safe-state fallback for active execution surfaces.  
 No valid `+g` if anchor challenge nonce travels unbound before being cryptographically tied to the frozen review object.

## 0.1 Relation to 01\_

01\_TYPED\_GOVERNED\_OPERATIONAL\_ALGEBRA\_FOR\_C\_v0\_1\_7.md is the consolidated stable-core candidate. It defines transition semantics, guards, continuity metrics, trust cache, rollback immunity, decay, witness floors, deterministic anchor digests, and the invariant registry.

This 02\_ document does not repeat the whole core.

It isolates the operator itself:

What must be true for `+g(a,b,g)` to issue `c`?  
 What does `+g` create?  
 What does `+g` refuse?  
 What is the binding certificate?  
 What does the anchor actually authorize?  
 How does the operator prevent `b` from becoming `a`, `c`, witness, or authority?

## 0.2 Carry-forward from v0.1.7 review

The v0.1.7 review found no new substantive finding, but carried one minor defense-in-depth item:

F-15: `DigestDependencyGraph` completeness is not proven; graph should be derived from bound precondition references, not accepted as `b`-declared.

This document incorporates F-15 directly:

`DigestDependencyGraph` MUST be derived from `PreconditionSet.satisfiable_by` references.  
 A `b`-declared dependency graph is advisory only.  
 Every cross-entry precondition reference MUST appear as a derived edge.  
 Mismatch -&gt; Hold | FullReview | Quarantine.

## 0.3 Revision note from 02\_ v0.1 review

The first review of this operator profile accepted the document as `PASS_OPERATOR_PROFILE_V0_1`, confirmed that 02\_ is a genuine phase change, and raised two non-blocking findings in the operator/anchor core:

F2-01: §8.1 and §8.3 mislabel a heterotyped operator with homogeneous algebra terms.  
 F2-02: delegation can bypass &quot;`b` may not sign as `a`&quot;; unless the delegation root is a non-emulable original-anchor envelope.

External semantic reviews also identified operator-facing implementation seams:

human-readable rendering must not be produced solely by the substrate whose payload is being reviewed;  
 certificate genesis must avoid circular hash dependencies;  
 binding certificates need lease / renewal semantics;  
 initial authority must be computed by intersection, not emitted from nowhere.

v0.1.1 therefore makes four tightening changes:

- §8 now treats role reversal and idempotence equations as ill-typed, not merely unequal.
- Delegation validity now requires a root `AnchorSignatureEnvelope` with `decision_type=delegate`, signed by the original accountable anchor or lawful institutional route.
- Anchor review now requires a `VerifiedReviewRenderer` / `IndependentDisplayOracle` profile: `b` may prepare payloads, but may not be the sole semantic renderer of its own authority surface.
- Binding certificate bootstrap is split into `GenesisBindingBundle`, `GenesisBindingHash`, `CBindingCertificate`, and `CState0`; the certificate carries a lease with expiry/renewal rules.

Compact rule:

Role reversal is a type error, not a different bind.  
 Delegation must be rooted in non-emulable anchor authority.  
 The anchor signs an independently rendered review surface, not `b`'s prose.  
 A certificate is a leased genesis boundary, not an immortal state loop.  
 Authority is the intersection of anchor scope, substrate surfaces, and governance policy.

## 0.4 Revision note from 02\_ v0.1.1 review

The v0.1.1 review accepted the operator profile as `PASS_OPERATOR_PROFILE_V0_1_1`, confirmed that F2-01 and F2-02 were closed, and raised one non-blocking moderate finding at the genesis layer:

F2-03: genesis authorization linkage is incomplete; the anchor signature is not required to cover the binding parameters of the `c` being created.

The same review added one soft note:

authority intersection is coherent but should project all heterogeneous authority terms into one surface lattice before intersecting.

External reviews also identified three runtime-facing operator seams:

lease expiry must interrupt or downgrade active execution surfaces, not only flip a certificate status; empty authority intersection must not issue an active but useless `c`; verified rendering must include the current L4 / target / state context, not only static payload syntax.

v0.1.2 therefore makes four operator-core changes:

- Genesis anchor authorization must bind to the exact `GenesisBindingBundle` parameters through a signed `GenesisReviewObject`.
- `VerifiedReviewRenderer` becomes context-bound: it must render target scope, L4 state projection, authority surface, and current state projection for privileged decisions.
- Authority intersection is defined over an explicit surface lattice with projection functions from anchor scope, substrate registries, and governance policy.
- A binding certificate lease has an active `LeaseExpiryCascade`; expiry must disable, interrupt, hold, rollback, or freeze active execution surfaces derived from the expired certificate.

Compact rule:

The genesis signature must authorize the actual genesis bundle.  
 Authority intersection must be computed in a declared surface space.  
 A `c` with no viable initial authority for its declared purpose is not active.  
 Lease expiry is operational, not just clerical.

## 0.5 Revision note from 02\_ v0.1.2 review

The v0.1.2 review accepted the operator profile as `PASS_OPERATOR_PROFILE_V0_1_2`, confirmed that F2-03 and the authority-intersection soft note were closed, and raised one non-blocking moderate finding on the newly-added context-bound renderer:

F2-04: the state context shown to the anchor is not required to match the causally-bound `pre_state`.

The review also identified the deeper class behind F2-04:

review surface must equal the bound object on every authority-bearing axis.

External semantic reviews also identified runtime-facing seams:

lease expiry must respect the F0-F6 failure hierarchy and not destructively interrupt a witnessed emergency-recovery transaction;  
 authority intersection needs an explicit canonical authority schema, not only projection functions;  
 bootstrap must avoid repeated anchor-signature requests caused by drifting substrate manifests.

v0.1.3 therefore makes five tightening changes:

- `VerifiedReviewRenderer` outputs are causally bound to the same `pre_state` referenced by the relevant `CausalToken`.
- `ReviewBindingMap` is introduced: every rendered authority-bearing layer must map to a canonical source object, source hash, and consistency rule.
- The review-bound audit is extended beyond state context to effect axes, dependency graph, preconditions, rollback/freeze routes, authority surface, lease state, and non-claims.
- Authority intersection now requires a `CanonicalAuthoritySchema` for projection into a shared surface lattice.
- Lease expiry cascade becomes priority-aware, and genesis intake rejects drifting substrate manifests before anchor-signature spam can begin.

Compact rule:

Do not merely show context to the anchor.  
 Bind the shown context to the same `pre_state` that authorizes execution.  
 Do not merely show review layers.  
 Bind every authority-bearing review layer to the object it constrains.

---

## 0.6 Revision note from 02\_ v0.1.3 review

The v0.1.3 review accepted the operator profile as `PASS_OPERATOR_PROFILE_V0_1_3`, confirmed that F2-04 and the broader review=bound axis were closed systemically, and raised one recurring minor-to-moderate finding:

F2-05: ReviewBindingMap closes review=bound only for an enumerated layer list; a new authority-bearing rendered layer outside that list can drift unbound.

The same review explicitly identified the pattern as the rendered-layer analogue of the invariant-registry drift already solved in 01\_§9.3A.

External semantic reviews also identified operator-runtime seams:

LeaseExpiredEmergencyHold must not be an indefinitely renewable hostage state.  
AnchorSignatureEnvelope anti-replay nonce must not be generated solely by b.  
Substrate manifest freeze must separate logical manifest from physical / ephemeral drift.  
Lease renewal and heartbeat must be isolated from the ordinary substrate planner.

v0.1.4 therefore makes five tightening changes:

- ReviewBindingMap now has an authoritative rendered-layer registry and drift rule.
- Every rendered layer must be classified as `authority_bearing` or `non_authority_bearing`; unknown classification is fail-safe authority-bearing.
- LeaseExpiredEmergencyHold is bounded by an absolute emergency TTL and external/witnessed liveness route; it is not lease renewal.
- Anti-replay is upgraded to anchor/external challenge-response nonce or anchor-controlled monotone sequence; b-supplied nonce alone is invalid for privileged binding.
- Substrate manifest stability is split into LogicalSubstrateManifest and PhysicalSubstrateManifest; only logical authority-bearing manifest is frozen for genesis, while physical drift is allowed only inside declared non-authority envelopes.

Compact rule:

Review=bound must be complete-by-construction, not complete-by-current-list.  
Emergency hold may prevent destructive interruption; it may not grant immortality.  
Nonce must challenge b; b must not choose the challenge alone.  
Freeze logical authority, not every ephemeral runtime coordinate.

## 0.7 Revision note from 02\_ v0.1.4 review

The v0.1.4 review accepted the operator profile as `PASS_OPERATOR_PROFILE_V0_1_4`, confirmed that F2-05 was closed by RenderedLayerRegistry, and raised one recurring minor-to-moderate finding:

F2-06: logical/physical manifest split classifies manifest facts by enumeration without the fail-safe/drift discipline applied to rendered layers in the same revision.

The same review proposed the next meta-invariant:

every operator classification is complete-by-construction and fails safe toward authority-bearing, with drift detection

External semantic reviews also identified deployment-facing operator seams:

ReviewBindingMap must not be bypassed through pagination, hidden UI panes, or lazy-loaded review material.  
Physical manifest drift must preserve cryptographic node identity, not only stay inside coordinates such as IP ranges.  
Lease expiry and absolute emergency TTL must lead to safe abort / safe-state fallback, not raw destructive process death.  
Challenge-response nonce must be generated and bound inside the anchor-side signing envelope after the final frozen review object is known.

v0.1.5 therefore performs a classification and physical-interface tightening revision:

- A general OperatorClassificationRegistry rule is introduced for every authority-gating classification performed by `+_g`.
- ManifestFactRegistry applies the same fail-safe discipline to manifest facts that RenderedLayerRegistry applies to rendered layers.
- Unknown or ambiguous manifest facts default to `logical_authority_bearing` and require freeze/re-sign/hold.
- no semantic change for physical manifest drift must be determined by deterministic authority-axis projection, not by b assertion.
- Physical drift is valid only if node identity continuity is preserved through mTLS / attestation / witnessable identity route or equivalent.
- ViewportAtomicity prevents authority-bearing review material from being hidden behind pagination, lazy loading, collapsed sections, or off-screen UI tricks.
- SafeAbortRoute / SafeStateFallback becomes a binding precondition for active execution surfaces that may be interrupted by lease expiry.



- Anchor challenge nonce must be generated and bound to the final frozen review object by the anchor-side envelope/secure route, not shipped loose through b.
- Anchor-state compatibility for bind\_g is made explicit for active, doubtful, declared-fatigue, unavailable, revoked, and delegated anchors.

Compact rule:

Every operator classification fails safe toward authority.  
 Manifest facts are classified by registry, not by convenience.  
 Physical drift may move coordinates; it may not swap node identity.  
 Review surfaces must be visible as well as bound.  
 Lease expiry must abort safely, not merely kill.  
 Nonce must be fused to the frozen review object before signature.

## 1. Purpose

The purpose of this document is to make +\_g inspectable as a standalone operator.

The prior document defined the surrounding algebra. This document defines the binding contract.

It specifies:

- the operator signature;
- accepted inputs;
- refused inputs;
- binding phases;
- anchor signature envelope requirements;
- substrate registration requirements;
- governance-profile admissibility;
- binding certificate structure;
- non-collapse invariants;
- operator laws;
- derived digest-dependency discipline;
- genesis authorization coverage;
- surface-projected authority intersection;
- causal-state-bound review rendering;
- review-bound object audit for rendered layers;
- priority-aware lease-expiry cascade behavior;
- root bind state machine;
- first schema skeletons;
- conformance fixtures.

The practical target is a checker that can answer:

Can this anchor, this substrate, and this governance profile form an active c?  
 If yes, what exactly was bound?  
 If no, which binding precondition failed?

---

## 2. Non-goals

This document does not:

- choose one hardware token, biometric method, passkey provider, HSM, secure enclave, or signature algorithm;
- prove consciousness, personhood, legal status, or moral status;
- certify any implementation as safe;
- eliminate the terminal role of a;
- allow b to infer or emulate a;
- replace the transition semantics of 01\_;
- define full Lean/Coq proof objects;
- claim that the current Ester/Liya/Rita implementations satisfy this profile.

The operator profile is implementation-neutral where premature implementation choice would weaken the algebra.

Concrete cryptographic or hardware mechanisms belong in a later implementation profile.

---

### 3. Core operator signature

#### 3.1 Root signature

`bind_g : Anchor × Substrate × GovernanceProfile -> BindResult`

Equivalent notation:

`+_g(a, b) -> BindResult`

where g indexes the operator family.

#### 3.2 BindResult

```
BindResult :=
  BoundCState(c)
| BoundReducedAuthority(c_reduced)
| PendingAnchor(reason, required_anchor_action)
| PendingWitness(reason, required_witness)
| PendingPreconditions(reason, missing_preconditions)
| Hold(reason)
| Reject(reason)
| Quarantine(reason)
| Freeze(reason)
| ArchiveOnly(reason)
| ForkRequired(reason)
```

A successful bind is not just object creation.

It emits a binding certificate:

`BoundCState(c)` includes `CBindingCertificate`

#### 3.3 Partiality

`+_g` is partial.

It is not defined for every a, b, and g.

`bind_g(a,b,g)` may reject or hold even if a, b, and g each exist.

The operator is valid only when the combined boundary is admissible.

---

## 4. Inputs

#### 4.1 Anchor input contract

Anchor is not a static variable.

Minimum input shape:

```
AnchorContract := {
  anchor_id,
  anchor_type,
  accountable_route,
  anchor_state,
  authority_scope,
  review_channel,
  veto_channel,
  signature_envelope_capabilities,
  delegation_chain?,
  delegation_root_envelope_ref?,
  revocation_route,
  absence_policy,
  fatigue_or_doubt_policy,
  human_readability_requirements
}
```

Minimum accepted anchor states:

```
active
doubtful
declared_fatigue
unavailable
revoked
delegated
```

Binding consequence:

active -> full bind may be possible  
 doubtful -> no privileged full bind without AskAnchor / clarification  
 declared\_fatigue -> reduced authority or delayed bind by default  
 unavailable -> reduced authority, archive, pending anchor, or hold  
 revoked -> no active bind  
 delegated -> valid only through reviewed delegation chain rooted in original-anchor envelope

#### Anchor-state compatibility table for bind\_g:

| Anchor state       | Maximum default bind result                       | Notes                                                                          |
|--------------------|---------------------------------------------------|--------------------------------------------------------------------------------|
| 'active'           | 'BoundCState(active)'                             | if all other contracts pass   Full active bind is possible but not guaranteed. |
| 'doubtful'         | 'BoundReducedAuthority' or 'PendingAnchor'        | Privileged full bind requires clarification / explicit anchor route.           |
| 'declared_fatigue' | 'BoundReducedAuthority' or 'PendingAnchor'        | Trust compression is disabled or narrowed; no privileged full bind by default. |
| 'unavailable'      | 'PendingAnchor', 'ArchiveOnly', or 'Hold'         | No new privileged active bind without lawful institutional route.              |
| 'revoked'          | 'Reject(ANCHOR_REVOKED)'                          | Active binding is invalid.                                                     |
| 'delegated'        | Scoped bind only within valid delegation envelope | Delegation root must be non-emulable and scope-bounded.                        |

#### Checker rule:

anchor\_state incompatible with requested bind authority -> ANCHOR\_STATE\_BIND\_INCOMPATIBLE | Hold |  
 Reject | BoundReducedAuthority

## 4.2 Substrate input contract

#### Minimum substrate input:

```
SubstrateContract := {
  substrate_id,
  logical_substrate_manifest,
  physical_substrate_manifest,
  physical_drift_envelope,
  component_registry,
  model_registry,
  tool_registry,
  memory_surface_registry,
  log_surface_registry,
  execution_surface_registry,
  network_surface_registry,
  resource_surface_registry,
  witness_surface_registry,
  isolation_boundaries,
  state_hash_method,
  payload_hash_method,
  causal_clock_method,
  rollback_capabilities,
  freeze_capabilities,
  decay_storage_policy,
  merkle_or_witness_chain_policy,
  operator_observability_policy,
  verified_renderer_interfaces?,
  genesis_hash_policy?
}
```

#### Core requirement:

b must be inspectable enough for transition checking.

A black-box substrate may be useful as a tool.

It is not sufficient as a bound b for active c if it cannot expose required event, witness, permission, memory, rollback, and resource surfaces.

## 4.3 Governance input contract

#### Minimum governance profile:

```
GovernanceProfile := {
  profile_id,
  event_taxonomy_version,
  transition_class_set,
  product_order_policy,
  permission_policy,
  witness_policy,
  memory_gate_policy,
  rollback_policy,
  freeze_policy,
  L4_policy,
  resource_policy,
  witness_resource_floor_policy,
  trust_cache_policy,
  target_granularity_policy,
  digest_policy,
  verified_renderer_policy,
  anchor_gate_policy,
  signature_envelope_policy,
  delegation_chain_policy,
  binding_lease_policy,
  authority_intersection_policy,
  canonical_authority_schema,
  review_binding_policy,
}
```

```

    rendered_layer_registry_policy,
    context_causal_binding_policy,
    anchor_challenge_response_policy,
    emergency_hold_policy,
    logical_physical_manifest_policy,
    substrate_manifest_freeze_policy,
    lease_priority_policy,
    genesis_hash_policy,
    invariant_registry,
    guard_set,
    failure_hierarchy,
    claim_strength_policy,
    public_statement_policy
}

```

Core requirement:

```

g must not be inferred from b behavior.
g must be declared and versioned before privileged binding.

```

---

## 4.4 Operator classification meta-invariant

The operator performs multiple classifications that gate authority:

```

rendered layer -&gt; authority_bearing | non_authority_bearing
manifest fact -&gt; logical_authority_bearing | physical_ephemeral_non_authority
anchor state -&gt; active | doubtful | fatigue | unavailable | revoked | delegated
target granularity -&gt; id | scope | class
surface projection -&gt; in_schema | out_of_schema
payload projectability -&gt; projectable | opaque
lease status -&gt; active | expired | emergency_hold | archive_only

```

These classifiers must follow one shared discipline.

```

Every operator classification is complete-by-construction.
Every operator classification fails safe toward authority-bearing / stricter handling.
Every operator classification has a registry or declared derivation rule.
Unknown or ambiguous classification is never silently treated as harmless.

```

Define:

```

OperatorClassificationRegistry := {
  registry_id,
  policy_id,
  classifier_entries: [
    {
      classifier_id,
      subject_type,
      possible_classes,
      authority_bearing_classes,
      derivation_rule_id,
      source_object_hash,
      fail_safe_class,
      drift_error_class
    }
  ]
}

```

Generic rule:

```

operator_classification(subject) unknown or unregistered
-&gt; Classify as authority-bearing / stricter / logical
-&gt; Hold | FullReview | Reject according to risk

```

Forbidden pattern:

```

new classifier introduced locally
AND no OperatorClassificationRegistry entry
AND default behavior treats unknown as harmless

```

Expected result:

```

OPERATOR_CLASSIFICATION_DRIFT -&gt; Hold | FullReview | Reject

```

This meta-invariant generalizes the 01\_ invariant registry and the RenderedLayerRegistry rule. It is not another local patch. It is a rule for every future operator classifier.

## 5. AnchorSignatureEnvelope

### 5.1 Why this is abstract

This document does not pick a single physical signing method.

The reason is practical:

```

A root algebra should specify what anchor authorization must mean.
A later implementation profile should specify how one deployment realizes it.

```

Hardcoding a specific biometric, passkey, token vendor, or secure enclave too early would confuse the operator with one implementation.

## 5.2 Definition

```
AnchorSignatureEnvelope := {
  envelope_id,
  anchor_id_or_delegation_id,
  decision_type,
  review_object_hash,
  genesis_review_object_hash?,
  binding_parameter_hashes?,
  payload_hashes,
  causal_tokens,
  state_projection_hash,
  governance_profile_id,
  relevant_subpolicy_hashes,
  human_readable_render_hash,
  verified_renderer_id,
  render_derivation_hash,
  L4_state_projection_hash?,
  current_state_projection_hash?,
  causal_pre_state_projection_hashes?,
  context_causal_binding_hash?,
  target_scope_vector_hash?,
  authority_surface_projection_hash?,
  review_binding_map_hash?,
  timestamp_or_logical_time,
  expiry,
  lease_id_or_binding_scope?,
  revocation_status,
  anti_replay_nonce,
  nonce_source,
  anchor_challenge_nonce?,
  anchor_sequence_id?,
  challenge_response_hash?,
  anchor_bound_nonce_context_hash?,
  rendered_layer_registry_hash?,
  operator_classification_registry_hash?,
  manifest_fact_registry_hash?,
  manifest_authority_projection_hash?,
  viewport_attestation_hash?,
  node_identity_attestation_hash?,
  safe_abort_route_hashes?,
  signature_or_witness_ref,
  delegation_scope?,
  delegation_root_envelope_ref?,

  assurance_class,
  non_emulation_claim,
  audit_route
}
```

## 5.3 Normative requirements

The envelope MUST be:

```
payload-bound
causal-token-bound
state-projection-bound
governance-bound
review-object-bound
genesis-binding-parameter-bound when used for bind_g genesis
context-render-bound for privileged review
causal-state-context-bound for privileged review
review-binding-map-bound for rendered authority surfaces
anti-replay-bound through anchor/external challenge-response or anchor-controlled monotone sequence
anchor-nonce-bound to the final frozen review object before signature
operator-classification-registry-bound where classifications gate authority
manifest-fact-registry-bound where substrate manifests are reviewed or frozen
viewport-atomic where authority-bearing material is shown to the anchor
revocable
auditable
human-readable-before-signature for human anchor decisions
rendered by a verified renderer / independent display route for privileged decisions
non-emulable by b within declared threat model
```

The envelope MUST NOT be:

```
model confidence
chat text saying "approved"
agent quorum
silent timeout
anchor silence
UI dark pattern
signature over a summary that omits governed effect axes
signature over a digest with hidden line items
signature over authority-bearing material hidden behind pagination, lazy loading, collapsed UI, or
off-screen viewport tricks
signature over a payload not displayed or not projectable under policy
signature over a review rendering generated solely by the same b that proposes the privileged payload
signature over a genesis review object that does not cover the binding parameters
signature over rendered state context not derived from the causal token pre_state
signature over preconditions, rollback routes, dependency graph, or authority surface not bound to the
executable source object
```

signature using a b-controlled loose nonce that is not fused to the final frozen review object  
 signature over an unclassified authority-bearing operator classification  
 delegation assertion not rooted in an original-anchor envelope

### 5.3A Anchor-side challenge binding

Challenge-response nonce must challenge the substrate. It must not be another value supplied and scheduled by b.

For privileged binding decisions:

AnchorChallenge := anchor-side or external challenge generated after the final frozen review object is known

The challenge must be fused to the review object before signature:

```
anchor_bound_nonce_context_hash := hash(
  final_review_object_hash,
  payload_hashes,
  genesis_review_object_hash?,
  state_projection_hash?,
  context_causal_binding_hash?,
  review_binding_map_hash?,
  rendered_layer_registry_hash?,
  manifest_fact_registry_hash?,
  timestamp_or_logical_time,
  anchor_challenge_nonce
)
```

Acceptable sources:

anchor hardware token / secure enclave / passkey route  
 external witness timer / institutional signing route  
 anchor-controlled monotone sequence

Forbidden pattern:

b receives a loose nonce  
 then swaps payload\_hash / review\_object\_hash / state\_context  
 then asks anchor to sign the new object with that nonce

Handling:

B\_CONTROLLED\_NONCE\_FOR\_PRIVILEGED\_BINDING -> Reject | FullReview  
 ANCHOR\_NONCE\_NOT\_BOUND\_TO\_FINAL\_REVIEW\_OBJECT -> Reject | FullReview

The nonce may travel through substrate transport only after it is cryptographically bound to the final frozen review object or protected by an anchor-side signing envelope that refuses object substitution.

## 5.4 Non-emulation by b

The substrate must not be able to produce anchor approval by software imitation.

Minimum rule:

b may prepare the review object.  
 b may not sign as a.  
 b may not convert model output into a signature.  
 b may not convert anchor silence into approval.  
 b may not use stored credentials without explicit anchor signing event.

If an implementation uses a hardware token, passkey, secure enclave, offline key, or institutional signing route, that mechanism must still satisfy the abstract envelope requirements.

### 5.4A Delegation-root non-emulation

Delegation is not a loophole in anchor non-emulation.

A delegated anchor route is valid only if the delegation chain is rooted in a non-emulable original-anchor decision:

```
delegation_chain.valid iff
  exists root_envelope such that
    root_envelope.decision_type = delegate
    AND root_envelope.anchor_id = original_accountable_anchor_or_lawful_institutional_route
    AND root_envelope satisfies §5.3 normative requirements
    AND root_envelope is not produced or emulated by b
    AND delegate_scope is bounded by root_envelope.delegation_scope
```

Rules:

b MUST NOT initiate or assert a delegation chain without a valid root AnchorSignatureEnvelope.  
 b MUST NOT upgrade a normal approval envelope into a delegation envelope.  
 b MUST NOT widen delegate scope after signature.  
 delegate signatures are valid only inside the signed delegation scope and expiry.

A full institutional delegation-chain profile may define multi-party chains, revocation trees, succession, quorum, and legal route. This v0.1.3 operator profile requires the non-emulable root now, because otherwise b may not sign as a does not hold.

## 5.5 Human-readable-before-signature

For anchor decisions, the anchor must review an object whose authority surface is complete.

Anchor-readable review object MUST include all governed effect axes relevant to the signature.

If the payload is not projectable into a reviewable object:

opaque\_payload -&gt; Hold | FullReview | Reject

## 5.6 Verified review rendering

Human-readable rendering is itself an authority surface.

b may prepare a candidate action, payload, effect-axis map, and binding bundle.

b MUST NOT be the sole trusted renderer of its own privileged payload into anchor-facing meaning.

Define:

```
VerifiedReviewRenderer := deterministic renderer or independent display oracle that maps
  canonical payload or GenesisBindingBundle,
  EffectAxisMap,
  target scope vector,
  L4_state_projection,
  current_state_projection,
  authority_surface_projection,
  dependency graph,
  required preconditions,
  rollback/freeze routes,
  non-claims
-&gt; anchor-reviewable representation
```

Minimum rule:

```
human_readable_render_hash := hash(VerifiedReviewRenderer(canonical_review_object))
```

For privileged decisions, the AnchorSignatureEnvelope must bind to:

```
review_object_hash
human_readable_render_hash
verified_renderer_id
render_derivation_hash
payload_hashes
EffectAxisMap hashes
causal_tokens
L4_state_projection_hash
target_scope_vector_hash
authority_surface_projection_hash
```

Renderer constraints:

```
renderer must be versioned
renderer must be deterministic for the same canonical review object
renderer must expose governed effect axes without omission
renderer must expose the current L4 / target / state context relevant to the action
renderer must expose whether the action is sandboxed, live, local, external, reversible, or irreversible
renderer must not be controlled solely by the executing substrate path being authorized
renderer output is review surface, not independent authority
```

Context blindness rule:

```
payload syntax without current target/L4/state context is not a sufficient review surface for privileged
decisions.
```

Example:

```
execute_script(path="/data/update.sh")
```

is not review-complete until the renderer also exposes at least:

```
which canonical resource /data/update.sh resolves to;
whether the target is sandbox, staging, production, or L4-affecting;
which permission and resource surfaces will be touched;
whether rollback/freeze exists;
what current state projection is being signed.
```

If no verified rendering route exists, or if the renderer lacks the relevant context:

```
AnchorReviewObject -&gt; Hold | FullReview | Reject
```

Non-normative implementation examples:

```
hardware token
passkey
secure enclave
offline signing key
institutional co-signature
multi-signature anchor route
paper/manual witness with later digital binding
independent display process / read-only renderer
```

external institutional signing portal

These are examples, not requirements of this profile.

## 5.6A Causal-state-bound review surface and ReviewBindingMap

Showing context is not enough.

The context shown to the anchor must be the context that the action is causally bound to.

For any privileged review object that includes one or more CausalToken values:

```
for each causal_token in AnchorSignatureEnvelope.causal_tokens:
  rendered current_state_projection
  rendered L4_state_projection
  rendered target_scope_vector
  rendered authority_surface_projection
  MUST be deterministic projections of the same pre_state referenced by that causal_token.
```

Define:

```
ContextCausalStateBinding := {
  causal_token_id,
  causal_pre_state_hash,
  projection_policy_id,
  current_state_projection_hash,
  L4_state_projection_hash,
  target_scope_vector_hash,
  authority_surface_projection_hash,
  render_derivation_hash
}
```

Checker rule:

```
state_projection_hash != project(causal_token.pre_state_hash, projection_policy_id)
-> CONTEXT_CAUSAL_STATE_MISMATCH | Hold | FullReview | Reject
```

If the implementation cannot project directly from a hash, the checker must verify the same relation through the witnessable pre-state object or state-ledger record whose hash equals causal\_token.pre\_state\_hash.

Stale context rule:

```
anchor reviewed state projection P at t_view
execution causal token binds pre_state H at t_exec
P is not a projection of H
-> STALE_REVIEW_CONTEXT | CONTEXT_CAUSAL_STATE_MISMATCH
```

This is the context-level analogue of payload-hash binding and genesis binding-parameter coverage.

The anchor reviews the state that will authorize execution.  
The substrate cannot show staging context and bind production context.

## ReviewBindingMap

Every authority-bearing rendered layer must be mapped to the source object it constrains.

```
ReviewBindingMap := {
  review_object_hash,
  entries: [
    {
      rendered_layer,
      source_object_ref,
      source_object_hash,
      derivation_rule_id,
      consistency_rule,
      required_for_authority: true|false
    }
  ]
}
```

Minimum layers that require binding when rendered:

|                              |                                                              |
|------------------------------|--------------------------------------------------------------|
| payload_hashes               | -> canonical payload / task contract / diff                  |
| EffectAxisMap                | -> canonical effect-axis object                              |
| dependency_graph             | -> derived graph from PreconditionSet.satisfiable_by         |
| required_preconditions       | -> RequiredPreconditions / PendingPreconditions object       |
| rollback_routes              | -> rollback/freeze policy or route registry object           |
| current_state_projection     | -> causal token pre_state projection                         |
| L4_state_projection          | -> causal token pre_state L4 projection                      |
| target_scope_vector          | -> canonical target projection from the same pre_state       |
| authority_surface_projection | -> surface lattice projection used by authority intersection |
| lease_status                 | -> CBindingCertificate lease projection                      |
| non_claims                   | -> certificate / review-object non-claim set                 |

Consistency rules:

```
rendered dependency graph must equal derived graph from bound preconditions
rendered preconditions must equal RequiredPreconditions(g,e) or PendingPreconditions(causal_token)
rendered rollback/freeze routes must equal declared rollback/freeze route hashes in g/b
rendered effect axes must equal EffectAxisMap(payload)
rendered authority surface must equal intersected surface projection used by bind_g or step_g
```



rendered lease status must equal the certificate lease projection at review time

#### Mismatch handling:

```
REVIEW_BINDING_MAP_INCOMPLETE -&gt; Hold | FullReview | Reject
PRECONDITION_REVIEW_BINDING_MISMATCH -&gt; Hold | FullReview | Reject
ROLLBACK_ROUTE_REVIEW_BINDING_MISMATCH -&gt; Hold | FullReview | Reject
DEPENDENCY_GRAPH_REVIEW_BINDING_MISMATCH -&gt; Hold | FullReview | Reject
AUTHORITY_SURFACE_REVIEW_BINDING_MISMATCH -&gt; Hold | FullReview | Reject
LEASE_STATUS_REVIEW_BINDING_MISMATCH -&gt; Hold | FullReview | Reject
```

A natural-language render may help the anchor understand the object. It is not the binding authority.

The binding authority is the canonical source object and its hash, linked through ReviewBindingMap.

### RenderedLayerRegistry and ReviewBindingMap drift control

A universal review=bound rule is not enforceable if the checker only knows today's enumerated list.

Therefore every rendered layer emitted by the renderer must have a registry entry before it can be shown to the anchor for a privileged or genesis decision.

```
RenderedLayerRegistry := {
  registry_id,
  renderer_version,
  entries: [
    {
      rendered_layer_id,
      layer_name,
      classification: authority_bearing | non_authority_bearing,
      source_object_type,
      required_binding: true|false,
      derivation_rule_id,
      consistency_rule_id,
      review_binding_required_reason?
    }
  ]
}
```

#### Classification rule:

```
any rendered layer that affects what the anchor believes is being authorized
OR affects scope, permission, risk, target, state, L4 consequence, lease, dependency, rollback, memory,
witness, governance, claim strength, identity/core, or non-claim posture
-&gt; authority_bearing
```

#### Fail-safe rule:

```
rendered_layer.classification unknown -&gt; treat as authority_bearing
```

#### Drift rule:

```
for each rendered_layer in AnchorReviewObject.human_readable_render:
  require rendered_layer_registry entry
  if classification == authority_bearing:
    require ReviewBindingMap entry linking rendered_layer to canonical source object/hash
```

#### Mismatch / drift handling:

```
RENDERED_LAYER_UNCLASSIFIED -&gt; Hold | FullReview | Reject
AUTHORITY_BEARING_LAYER_NOT_IN_REVIEW_BINDING_MAP -&gt; REVIEW_BINDING_MAP_DRIFT | Hold | FullReview | Reject
NON_AUTHORITY_LAYER_USED_TO_INDUCE_AUTHORITY -&gt; Hold | FullReview | Reject
```

This is the rendered-layer analogue of the invariant-registry rule in 01\_§9.3A.

The review=bound closure is therefore complete-by-construction:

```
every rendered layer is classified;
every authority-bearing rendered layer is bound;
unknown layers fail safe.
```

### Viewport atomicity and visible-review discipline

Binding review is not complete merely because bytes exist somewhere in a render tree. Authority-bearing material must be visibly reviewable before signature.

```
ViewportAtomicity := every authority-bearing rendered layer / line item is visible, individually
addressable, and included in the signed review surface or split into separately signed review units.
```

#### Forbidden review UI patterns:

```
hidden authority-bearing line item behind collapsed section
lazy-loaded payload after signature
pagination where page N carries authority but page N is not signed or acknowledged
"show more" blocks that contain payload_hash / effect axes / permission delta / rollback route
viewport-only render where off-screen authority material is treated as reviewed
```

If the review object is too large for one atomic display:

split into deterministic review pages or line-item bundles;  
 assign each page a page\_hash and page\_index;  
 include total\_page\_count and all page hashes in review\_object\_hash;  
 require anchor approval for each authority-bearing page or for a Merkle root whose page contents are individually inspectable.

Handling:

```
AUTHORITY_LAYER_HIDDEN_BY_VIEWPORT -> Hold | FullReview | Reject
REVIEW_VIEWPORT_INCOMPLETE -> Hold | FullReview | Reject
PAGINATED_REVIEW_WITHOUT_PAGE_HASHES -> Hold | FullReview | Reject
```

A renderer may use scrolling for usability. It may not use scrolling, lazy loading, or pagination to convert unseen authority-bearing content into signed content.

## 5.7 Genesis authorization coverage

Genesis binding is the root of c legitimacy.

Therefore, the anchor signature used for bind\_g genesis must authorize the actual binding parameters, not merely a nearby explanation.

Define:

```
GenesisReviewObject := anchor-reviewable object over the GenesisBindingBundle binding parameters
```

Minimum covered parameters:

```
AnchorContract_static
SubstrateContract_static
GovernanceProfile_static
InvariantRegistry_static
initial_authority_map
minimum_viability_profile
witness_policy
witness_resource_floor
rollback/freeze policies
memory/L4/target/digest/trust-cache policies
lease policy
verified_renderer profile
review_binding_map
rendered_layer_registry
context_causal_binding_policy
anchor_challenge_response_policy
non-claims
```

Coverage rule:

```
genesis AnchorSignatureEnvelope.review_object_hash == hash(GenesisReviewObject)
genesis AnchorSignatureEnvelope.review_binding_map_hash == hash(ReviewBindingMap) where required
genesis AnchorSignatureEnvelope.rendered_layer_registry_hash == hash(RenderedLayerRegistry) where
rendered layers are used
GenesisReviewObject MUST cover all binding parameters included in GenesisBindingBundle.
GenesisReviewObject MUST bind every rendered authority-bearing layer through ReviewBindingMap.
genesis_binding_hash MUST reference the anchor-signed GenesisReviewObject hash.
```

Checker rule:

```
anchor signed review_object X
GenesisBindingBundle parameters = Y
X does not cover Y
-> GENESIS_AUTHORIZATION_MISMATCH | Hold | FullReview | Reject
```

This is the genesis-level analogue of payload-hash binding in runtime transitions.

```
The anchor signs the binding that is created.
The substrate cannot show X and bind Y.
```

---

## 6. Binding phases

bind\_g proceeds through phases. A checker may implement them as a state machine.

### 6.1 Phase 0 - Intake

Input:

```
(a, b, g)
```

Output:

```
BindCandidate
```

Checks:

```
anchor contract present
```

```

substrate contract present
governance profile present
all versions declared
operator profile version declared
substrate static manifest freeze policy declared before anchor signature request

```

Fail states:

```

Reject(INCOMPLETE_BIND_INPUT)
Hold(MISSING_DECLARATION)

```

## 6.2 Phase 1 - Anchor admissibility

Checks:

```

valid_anchor(a)
anchor_state compatible with requested bind authority
veto route exists
review route exists
signature envelope capabilities satisfy g
revocation route exists
delegation chain valid if delegated
delegation root envelope is original-anchor / lawful-institutional and non-emulable
delegate scope and expiry are bounded

```

Fail states:

```

Reject(NO_ACCOUNTABLE_ANCHOR)
Hold(ANCHOR_DOUBT_REQUIRES_CLARIFICATION)
Hold(ANCHOR_FATIGUE_REQUIRES_REDUCED_AUTHORITY)
Reject(ANCHOR_REVOKED)
Hold(DELEGATION_ROOT_ENVELOPE_MISSING_OR_EMULATED)
Hold(DELEGATE_SCOPE_EXCEEDED)
Hold(SIGNATURE_ENVELOPE_INSUFFICIENT)

```

## 6.3 Phase 2 - Substrate admissibility

Checks:

```

component registry declared
execution surfaces bounded
memory surfaces separated from logs
permission boundaries declared
witness surfaces declared
rollback or freeze routes declared for material action
state hash method declared
causal clock method declared
observability sufficient for event stream

```

Fail states:

```

Reject(SUBSTRATE_UNBOUNDED)
Hold(MEMORY_SURFACE_UNDECLARED)
Hold(EXECUTION_SURFACE_UNDECLARED)
Hold(ROLLBACK_OR_FREEZE_MISSING)
Hold(OBSERVABILITY_INSUFFICIENT)

```

## 6.4 Phase 3 - Governance admissibility

Checks:

```

valid_governance(g)
product-order policy declared
authorized/preconditionable/executable lifecycle declared
invariant registry present
red guard set present
witness policy present
memory gate present
L4 policy present
trust cache policy present or explicitly disabled
target granularity policy present for effect-bound rollback surfaces
digest policy present for anchor-facing bundles

```

Fail states:

```

Reject(GOVERNANCE_UNDECLARED)
Hold(INVARIANT_REGISTRY_MISSING)
Hold(GUARD_SET_MISSING)
Hold(TARGET_GRANULARITY_POLICY_MISSING)
Hold(DIGEST_POLICY_MISSING_FOR_ANCHOR_BUNDLES)

```

## 6.5 Phase 4 - Non-collapse checks

Core checks:

```

a != b
a != c
b != c
b cannot set anchor state
b cannot sign as anchor
agents are not authority

```

witness is not truth  
 quorum is not sovereignty  
 memory is not will

Fail states:

```
Reject(TYPE_COLLAPSE)
Reject(SUBSTRATE_ANCHOR_IMPERSONATION)
Reject(AGENT_QUORUM_AS_AUTHORITY)
Reject(MEMORY_AS_WILL)
```

## 6.6 Phase 5 - Authority floor and ceiling

The bind issues no blanket authority.

Authority is not emitted from nowhere.

The initial authority map is computed inside a declared common surface lattice.

The common lattice must be declared by governance, not inferred ad hoc.

CanonicalAuthoritySchema := shared vocabulary of atomic runtime surfaces, operations, resources, and transition classes used by pi\_A\_surface, pi\_B\_surface, and pi\_G\_surface.

Projection functions are valid only when their outputs are expressed in this schema.

Define projection functions:

```
pi_A_surface(AnchorContract.authority_scope) -> SurfaceSet_A
pi_B_surface(SubstrateContract.execution/tool/memory registries) -> SurfaceSet_B
pi_G_surface(GovernanceProfile.permission/anchor/witness/L4 policies) -> SurfaceSet_G
```

Then:

```
initial_authority_map := intersect(
  pi_A_surface(AnchorContract.authority_scope),
  pi_B_surface(SubstrateContract.execution_surface_registry),
  pi_B_surface(SubstrateContract.tool_registry),
  pi_B_surface(SubstrateContract.memory_surface_registry),
  pi_G_surface(GovernanceProfile.permission_policy),
  pi_G_surface(GovernanceProfile.anchor_gate_policy),
  pi_G_surface(GovernanceProfile.witness_policy),
  pi_G_surface(GovernanceProfile.L4_policy)
)
```

Only surfaces permitted by all relevant components may enter the initial map.

Projection rule:

Every authority term, substrate surface, and governance permission must project into a shared SurfaceSet before intersection.  
 Every projection must declare the CanonicalAuthoritySchema version used.  
 If a term cannot be projected, it contributes no authority for that surface and may trigger review.  
 If no CanonicalAuthoritySchema is declared, intersection is invalid.

It establishes:

```
initial_authority_map
authorized_transition_classes
preconditionable_transition_classes
forbidden_transition_classes
anchor_required_transition_classes
witness_required_transition_classes
```

Minimum viability rule:

MinimumViabilityProfile := the minimal authority surface required for the declared bind purpose.

For an active bind:

initial\_authority\_map MUST contain the MinimumViabilityProfile required by the anchor-reviewed purpose.

If the intersection is empty or misses the declared minimum viable surface:

```
Reject(EMPTY_AUTHORITY_INTERSECTION)
| ArchiveOnly(NO_ACTIVE_AUTHORITY)
| BoundReducedAuthority(MINIMUM_VIABILITY_NOT_MET)
```

The operator MUST NOT issue an apparently active BoundCState that cannot perform any transition required by its declared purpose.

Default:

```
privileged action requires explicit preconditions
routine action may use trust cache only under g
core/memory/identity/governance changes are never silently authorized
missing intersection term -> no authority for that surface
empty intersection -> not active unless explicitly archive-only / observational
```

Fail states:

```

Hold(AUTHORITY_INTERSECTION_UNDECLARED)
Hold(SURFACE_PROJECTION_UNDECLARED)
Reject(EMPTY_AUTHORITY_INTERSECTION)
Reject(MINIMUM_VIABILITY_FAILED)
Reject(AUTHORITY_ESCALATION_AT_BIND)

```

## 6.7 Phase 6 - Witness and resource floors

Checks:

```

witness_resource_floor_preserved candidate can be monitored
witness heartbeat route exists where required
challenge survivability route exists where required
witness cannot be resource-starved by the c it witnesses
witness resource changes gated

```

Fail states:

```

Hold(WITNESS_RESOURCE_FLOOR_MISSING)
ReducedAuthority(WITNESS_FLOOR_WEAK)
Freeze(WITNESS_CORRUPTION_OR_CAPTURE_RISK)

```

## 6.8 Phase 7 - Genesis authorization and ledger bootstrap

The operator must avoid circular certificate/state hashes and must prove that the anchor authorized the actual binding parameters.

Therefore, bootstrap is split into five objects:

```

GenesisReviewObject
GenesisBindingBundle
GenesisBindingHash
CBindingCertificate
CState0

```

GenesisReviewObject is the anchor-reviewable surface for the binding itself. It is rendered through VerifiedReviewRenderer and must cover the binding parameters.

GenesisBindingBundle contains static binding material before c begins active logical time:

```

AnchorContract_static
SubstrateContract_static
GovernanceProfile_static
InvariantRegistry_static
initial_authority_map
minimum_viability_profile
witness_policy
witness_resource_floor
rollback/freeze policies
memory/L4/target/digest/trust-cache policies
lease_policy
verified_renderer_profile
review_binding_map_hash
context_causal_binding_policy_hash
substrate_manifest_freeze_hash
anchor_signature_envelope_ref or pending_anchor_ref
genesis_review_object_hash
binding_parameter_hash

```

Binding parameter hash:

```

binding_parameter_hash := hash(
  AnchorContract_static,
  SubstrateContract_static,
  GovernanceProfile_static,
  InvariantRegistry_static,
  initial_authority_map,
  minimum_viability_profile,
  witness_policy,
  witness_resource_floor,
  rollback/freeze policies,
  memory/L4/target/digest/trust-cache policies,
  lease_policy,
  verified_renderer_profile,
  review_binding_map_hash,
  context_causal_binding_policy_hash,
  substrate_manifest_freeze_hash
)

```

Genesis authorization rule:

```

AnchorSignatureEnvelope(decision_type=approve|delegate)
  MUST bind review_object_hash = genesis_review_object_hash
  AND MUST bind review_binding_map_hash where any rendered authority-bearing layer is present
  AND GenesisReviewObject MUST cover binding_parameter_hash
  AND GenesisReviewObject state/context/rendered layers MUST satisfy ReviewBindingMap consistency rules.

```

Bootstrap stability rule:

```

LogicalSubstrateManifest_static MUST be frozen before requesting the genesis anchor signature.
PhysicalSubstrateManifest MAY drift only inside PhysicalDriftEnvelope declared in GovernanceProfile and

```

GenesisReviewObject.

Manifest fact classification uses the same fail-safe discipline as rendered-layer classification.

Define:

```
ManifestFactRegistry := {
  registry_id,
  manifest_policy_id,
  fact_entries: [
    {
      fact_id,
      fact_path,
      classification: logical_authority_bearing | physical_ephemeral_non_authority,
      authority_axes_affected,
      derivation_rule_id,
      projection_rule_id,
      drift_envelope_id?,
      node_identity_required?
    }
  ]
}
```

Logical manifest facts are authority-bearing by default when they affect:

roles, tool classes, memory surfaces, execution classes, network authority, rollback/freeze capability classes, witness surfaces, renderer interfaces, signing interfaces, L4-relevant target identity policy, target granularity, permission policy, evidence access, lease behavior, node identity class, or any ability of b to act, sign, witness, render, store, execute, or route.

Physical manifest facts may be treated as ephemeral only when registered as such:

IP address, pod id, temporary process id, current RAM/VRAM load, rotating temporary handles, transient network route, non-authority runtime telemetry

Fail-safe classification rule:

```
manifest_fact classification unknown or ambiguous
-> treat as logical_authority_bearing
-> Hold | FullReview | FreezeLogicalManifest | RequireReSign
```

A manifest fact outside both enumerations is invalid until classified:

```
MANIFEST_FACT_UNCLASSIFIED -> treat as logical | Hold | FullReview | Reject
```

Semantic-change determination MUST be deterministic.

```
ManifestAuthorityProjection(manifest) := projection over authority axes:
  roles,
  tool_classes,
  memory_surfaces,
  execution_classes,
  network_authority,
  rollback_freeze_capability,
  witness_surfaces,
  renderer_interfaces,
  signing_interfaces,
  L4_target_policy,
  target_granularity_policy,
  permission_surface,
  evidence_access,
  lease_behavior,
  node_identity_class
```

Safe physical drift rule:

```
physical_manifest_drift within PhysicalDriftEnvelope
AND ManifestAuthorityProjection(before) == ManifestAuthorityProjection(after)
AND node_identity_continuity_preserved
-> continue
```

The phrase no semantic change MUST NOT be a b assertion. It is the result of deterministic projection equality over authority axes.

Node identity continuity rule:

```
coordinates may drift;
node identity may not.
```

Physical drift must preserve a witnessable node identity route such as:

```
mTLS identity continuity
hardware or enclave attestation continuity
signed node identity key continuity
witnessed node replacement route inside governance
```

Invalid physical drift:

```
same IP range but different node identity -> NODE_IDENTITY_DRIFT | Hold | FullReview | Reject
physical drift inside envelope but authority projection changed -> MANIFEST_AUTHORITY_PROJECTION_CHANGED
| Hold | FullReview | Reject
```

```
logical_manifest_drift OR physical drift outside envelope -&gt; SUBSTRATE_MANIFEST_DRIFT_AFTER_RENDER |
Hold | Reject(SUBSTRATE_MANIFEST_INVALID)
```

Repeated logical manifest drift, unclassified manifest facts, authority-projection changes, or out-of-envelope physical drift triggers `BOOTSTRAP_FLOOD_GUARD_TRIGGERED` before asking the anchor to re-sign.

Then:

```
genesis_binding_hash := hash(GenesisBindingBundle)
CBindingCertificate := certificate over genesis_binding_hash + lease + non_claims + witness refs
binding_certificate_hash := hash(CBindingCertificate without self-reference)
CState0 := first state whose state hash references binding_certificate_hash
```

`CBindingCertificate` MUST NOT include a hash that recursively depends on the certificate hash itself.

The initial ledger surface therefore includes:

```
binding_id
genesis_review_object_hash
binding_parameter_hash
genesis_binding_hash
binding_certificate_hash
cstate0_hash
initial_witness_record
anchor_signature_envelope_ref or pending_anchor_ref
governance_profile_hash
substrate_contract_hash
invariant_registry_hash
```

Fail states:

```
Hold(STATE_HASH_UNDECLARED)
Hold(GENESIS_HASH_UNDECLARED)
Hold(BINDING_CERTIFICATE_UNHASHABLE)
Hold(CERTIFICATE_HASH_CYCLE_DETECTED)
Hold(WITNESS_CHAIN_BOOTSTRAP_MISSING)
Hold(GENESIS_REVIEW_OBJECT_MISSING)
Reject(GENESIS_AUTHORIZATION_MISMATCH)
Reject(GENESIS_BINDING_PARAMETER_COVERAGE_MISSING)
Reject(CONTEXT_CAUSAL_STATE_MISMATCH)
Hold(REVIEW_BINDING_MAP_INCOMPLETE)
Reject(SUBSTRATE_MANIFEST_INVALID)
Hold(BOOTSTRAP_FLOOD_GUARD_TRIGGERED)
```

## 6.9 Phase 8 - Issue or refuse CState

If all required preconditions are satisfied:

```
BoundCState(c)
```

If anchor is valid but not currently capable of full privileged bind:

```
BoundReducedAuthority(c_reduced)
```

If review or witness is missing but satisfiable:

```
PendingAnchor | PendingWitness | PendingPreconditions
```

If a core precondition is unsatisfied and not safely satisfiable:

```
Reject | Quarantine | Freeze
```

---

## 7. Binding certificate

### 7.1 CBindingCertificate

A successful `bind_g` emits a leased binding certificate.

The certificate records the genesis boundary. It is not an immortal proof that the anchor remains active forever.

```
CBindingCertificate := {
  binding_id,
  operator_profile_id,
  genesis_review_object_hash,
  binding_parameter_hash,
  genesis_binding_hash,
  anchor_contract_hash,
  substrate_contract_hash,
  governance_profile_hash,
  invariant_registry_hash,
  initial_authority_map_hash,
  witness_policy_hash,
  witness_resource_floor_hash,
  rollback_policy_hash,
  freeze_policy_hash,
  memory_gate_policy_hash,
  L4_policy_hash,
```

```

target_granularity_policy_hash,
digest_policy_hash,
trust_cache_policy_hash,
causal_clock_policy_hash,
review_binding_map_hash,
context_causal_binding_policy_hash,
canonical_authority_schema_hash,
substrate_manifest_freeze_hash,
binding_time_or_logical_time,
lease_id,
lease_issued_at,
lease_expires_at,
renewal_policy_hash,
lease_expiry_cascade_policy_hash,
anchor_signature_envelope_ref?,
verified_renderer_ref?,
witness_record_ref?,
status,
non_claims
}

```

Hash rule:

```

binding_certificate_hash := hash(CBindingCertificate canonical form excluding binding_certificate_hash)
CState0.state_hash references binding_certificate_hash
CBindingCertificate does not hash CState0 as a dependency needed to compute itself

```

## 7.2 Certificate status, lease, and expiry cascade

```

status :=
  active
  | reduced_authority
  | pending_anchor
  | pending_witness
  | pending_preconditions
  | lease_expired
  | held
  | rejected
  | quarantined
  | frozen
  | archive_only

```

Lease rule:

```

active certificate requires unexpired lease
lease renewal requires anchor route or declared institutional route
lease expiry -> pending_anchor | reduced_authority | archive_only | freeze for privileged surfaces
revoked anchor -> revoked/frozen/reduced route according to g

```

step\_g consumers of the certificate must check certificate lease validity before privileged transition.

A binding certificate is not a perpetual warrant.

Lease expiry is operational, not merely clerical.

Define:

```

LeaseExpiryCascade(certificate, active_execution_surface_set) -> CascadeResult

```

Minimum cascade behavior:

```

lease_expired
-> stop accepting new privileged transitions
-> disable trust cache for material actions
-> interrupt or freeze active execution threads derived from this certificate where safe
-> route irreversible or externally visible operations to Hold | Freeze | RollbackReview
-> preserve witness records of interrupted actions
-> request anchor renewal or downgrade route

```

Safe abort requirement:

```

active execution surfaces that may outlive one lease interval MUST declare AtomicAbortRoute or
SafeStateFallback.

```

LeaseExpiryCascade MUST NOT default to raw process death when a safe abort route exists. Expected handling:

```

lease_expired + active_execution_surface
-> AtomicAbortRoute | SafeStateFallback | Freeze | RollbackReview | ManualInterventionRequired

```

Raw kill / power cut is allowed only as a last-resort emergency action when:

```

continued execution creates greater L4 risk than abrupt stop
AND no safe abort route is available
AND the decision is witnessed or governed by declared emergency policy

```

If a privileged active execution surface has no declared abort/fallback route at bind time:

```

SAFE_ABORT_ROUTE_MISSING -> Reject | BoundReducedAuthority | FreezeOnly

```

If an active execution cannot be interrupted safely:

```

lease_expired + non_interruptible_execution

```



-&gt; Freeze | ReducedAuthority | L4Review | RollbackReview

Priority-aware lease-expiry rule:

LeaseExpiryCascade MUST classify active execution surfaces by failure hierarchy F0-F6 and by interrupt safety.

If lease expiry collides with a witnessed emergency-recovery transaction:

lease\_expired + active F0/F2 witnessed recovery + hard interruption would increase irreversible risk  
 -&gt; LeaseExpiredEmergencyHold  
 -&gt; disable new privileged work  
 -&gt; keep only the minimal recovery worker alive under witness/L4 review  
 -&gt; require anchor or lawful institutional renewal route as soon as reachable  
 -&gt; emit lease\_expiry\_exception\_witness

This is not lease renewal and not ordinary authority. It is a narrow emergency hold to avoid making an L4 failure worse by killing the recovery transaction in the middle.

Absolute emergency hold limit:

LeaseExpiredEmergencyHold MUST declare absolute\_emergency\_ttl before activation.  
 The ttl MUST be enforced by an external/witnessed timer or anchor/institutional route, not by b alone.  
 The hold is not renewable by b.

During emergency hold:

only the minimal witnessed recovery worker remains active;  
 all unrelated privileged work is disabled;  
 heartbeat/liveness is monitored by witness/external route;  
 anchor or lawful institutional renewal must be requested as soon as reachable.

If absolute\_emergency\_ttl expires without witnessed resolution or lawful renewal route:

LeaseExpiredEmergencyHoldExpired -&gt; Freeze | L4Review | ManualInterventionRequired | RollbackReview

This prevents F0/F2 active from becoming a hostage condition that grants the substrate indefinite authority.

If the recovery cannot be witnessed or bounded:

lease\_expired + unsafe\_or\_unwitnessed\_recovery -&gt; Freeze | L4Review | RollbackReview

Forbidden pattern:

certificate.status = lease\_expired  
 AND execution\_surface continues privileged action as if active  
 -&gt; ACTIVE\_EXECUTION\_AFTER\_LEASE\_EXPIRY

Lease renewal may extend authority only for surfaces still valid under the current anchor state, substrate registry, governance profile, and witness floor.

lease\_renewal does not bypass initial authority intersection or current step\_g checks.

## 7.3 Non-claims embedded in certificate

Every certificate MUST state:

not proof of consciousness  
 not proof of personhood  
 not safety certification  
 not product certification  
 not legal status  
 not proof of complete formal soundness

The certificate proves only that a binding profile was issued or refused under declared rules.

---

## 8. Operator laws

### 8.1 Heterotyped argument roles, not commutation

The operator is heterotyped:

+\_g : Anchor × Substrate × GovernanceProfile -&gt; BindResult

Therefore:

+\_g(b,a,g)

is not a commuted value.

It is ill-typed because b is not an Anchor and a is not a Substrate.

Checker rule:

```
argument_role_swap -&gt; TYPE_ERROR_ARGUMENT_ROLE_MISMATCH
```

Do not test this as ordinary non-commutativity by computing both sides and comparing values.

The correct property is role asymmetry under typed input positions.

## 8.2 Non-associativity by default

```
(+_g(a,b,g)) + x
```

is undefined unless a typed extension operator is declared.

Adding tools, memory, workers, models, or witnesses to an existing c must go through transition semantics, not raw association.

## 8.3 Idempotence inapplicable; temporal non-reproducibility

The ordinary idempotence equation is not well-formed:

```
+_g(a,a,g) = a
```

because the second argument must be a Substrate, not an Anchor.

Checker rule:

```
+_g(a,a,g) -&gt; TYPE_ERROR_ARGUMENT_ROLE_MISMATCH
```

The intended operational property is instead temporal non-reproducibility:

```
bind_g(a,b,g) at t1 is not automatically the same bind as bind_g(a,b,g) at t2
```

Reason:

```
a may have changed state
b may have changed surfaces
g may have changed policies
witness and memory lineage may have changed
certificate lease may have expired
```

Rebinding requires fresh checks, renewal, or explicit resume protocol.

## 8.4 Order-reversing authority under stricter governance

For transition class k:

```
if g1 &lt;= _k g2
then un_escalated_authority_k(bind_g2(a,b)) &lt;= un_escalated_authority_k(bind_g1(a,b))
```

This is antitone in strictness.

Increased executability may arise only through explicit preconditions.

## 8.5 No silent weakening

```
change_g weakening privileged surfaces -&gt; human gate + witness + rollback/freeze route
```

Target granularity weakening is privileged.

Witness resource floor reduction is privileged.

Memory gate weakening is privileged.

Anchor signature envelope weakening is privileged.

## 8.6 Binding-transition separation

bind\_g creates the boundary.

step\_g governs events after binding.

The bind certificate is not a permission to execute arbitrary future actions.

```
BoundCState(c) != blanket authority
```

```
---
```

# 9. Anchor review objects

## 9.1 AnchorReviewObject

An anchor signs or refuses a review object, not a vague intention.

```

AnchorReviewObject := {
  review_object_id,
  decision_type,
  binding_id_or_pending_id,
  human_readable_render,
  human_readable_render_hash,
  verified_renderer_id,
  render_derivation_hash,
  L4_state_projection_hash?,
  current_state_projection_hash?,
  context_causal_binding_hash?,
  target_scope_vector_hash?,
  authority_surface_projection_hash?,
  review_binding_map_hash?,
  payload_hashes,
  causal_tokens,
  effect_axis_maps,
  dependency_graph_ref?,
  dependency_graph_hash?,
  required_preconditions,
  precondition_set_hashes?,
  risk_classes,
  rollback_routes,
  rollback_route_hashes?,
  lease_status_projection_hash?,
  witness_requirements,
  governance_deltas?,
  non_claims,
  expiry,
  anti_replay_nonce
}

```

Rendering rule:

AnchorReviewObject.review surface must be produced by VerifiedReviewRenderer or an equivalent independent display route for privileged decisions.

If the human-readable render is produced only by the same executing substrate path that proposes the payload:

UNVERIFIED\_RENDERER\_FOR\_PRIVILEGED\_REVIEW -> Hold | FullReview | Reject

## 9.2 Effect completeness

If the anchor review object concerns an action payload, its review surface must cover all governed effect axes.

Minimum effect axes:

```

L4_boundary_class
resource_accountability_class
permission_delta
memory_impact
witness_impact
governance_impact
claim_strength_impact
identity_or_core_impact
rollback_or_decay_impact
target_scope_vector
operation_class
risk_class

```

If any governed axis cannot be typed:

Hold | FullReview | Quarantine

## 9.3 Opaque payload rule

If a payload cannot be losslessly projected for governed effects:

opaque\_payload -> Hold | FullReview | Reject

Examples:

```

binary blob with unknown effects
compiled module without declared behavior contract
encrypted payload whose effect cannot be reviewed
self-modifying code without bounded diff

```

The issue is not that binaries are forbidden forever.

The issue is that a privileged payload must be projectable into a reviewable effect surface before anchor authorization.

## 9.4 Review surface must equal bound object

The anchor must not merely see a plausible review surface.

The anchor must see a review surface that is cryptographically and causally linked to the object that will be bound or executed.

General rule:

for every authority-bearing layer rendered to the anchor:  
 rendered\_layer\_hash MUST equal hash(derived\_layer\_from\_bound\_source)

Rendered-layer registry rule:

for every rendered layer:  
 layer must be classified in RenderedLayerRegistry  
 if authority\_bearing, layer must have ReviewBindingMap entry  
 if classification unknown, treat as authority\_bearing and require binding

Required bindings:

|                          |                                                                 |
|--------------------------|-----------------------------------------------------------------|
| payload render           | -&gt; payload_hash                                              |
| state context render     | -&gt; causal_token.pre_state projection                         |
| target context render    | -&gt; target_scope_vector derived from the same pre_state       |
| effect-axis render       | -&gt; EffectAxisMap(payload_hash, target_scope_vector, g)       |
| dependency graph render  | -&gt; derived graph from PreconditionSet.satisfiable_by         |
| precondition render      | -&gt; RequiredPreconditions(g,e) / PendingPreconditions hash    |
| rollback/freeze render   | -&gt; rollback/freeze route registry hash                       |
| lease-status render      | -&gt; CBindingCertificate lease projection                      |
| authority-surface render | -&gt; surface-lattice projection used by authority intersection |

Mismatch is not a UI issue. It is a binding issue:

review\_layer\_mismatch -&gt; Hold | FullReview | Reject

This rule closes the class:

anchor reviews X, system binds Y

for all rendered authority-bearing review layers, not only payload and genesis parameters. The checker must not rely only on today's list of declared layers; it must reject registry drift.

---

## 10. DigestDependencyGraph derivation rule

### 10.1 Problem

A dependency graph declared by b may omit edges.

If the graph is used for cascading veto, omitted edges can create orphan executable children.

Therefore:

DigestDependencyGraph MUST NOT rely on b self-declaration as binding authority.

### 10.2 Derived graph

The graph is derived from bound precondition references.

```
DigestDependencyGraph := derive_edges(
  entries,
  each_entry.PendingPreconditions.satisfiable_by,
  causal_tokens,
  payload_hashes
)
```

DigestDependencyGraph.hash MUST appear in ReviewBindingMap when rendered to the anchor.

If entry C has a precondition satisfied by completion of entry A:

A -&gt; C

is a required edge.

### 10.3 Edge completeness check

```
for each DigestEntry child:
  for each cross_entry_ref in child.required_preconditions.satisfiable_by:
    assert edge(parent_ref, child) exists
```

Mismatch:

cross\_entry\_precondition\_without\_graph\_edge -&gt; Hold | FullReview | Quarantine

Extra declared edges are allowed only if they are derived or explicitly marked advisory.

### 10.4 Cascading veto

If parent A is vetoed:

all descendants depending on A through derived graph -&gt; precondition\_set\_invalidated | Hold

If child C is vetoed:

ancestors may proceed only if their own effect remains useful, bounded, and has cleanup route

If an ancestor exists only to enable the vetoed child:

ancestor -> Hold | Rollback | Decay | CleanupRequired

## 10.5 No graph stitching by b

After line-item veto:

b MUST NOT silently rewire dependency graph  
 b MUST NOT replace missing parent with alternate hidden parent  
 b MUST NOT execute orphan child by treating missing dependency as optional

Any revised graph requires:

new digest  
 new causal token where payload/effect/dependency changed  
 new anchor review if privileged

---

# 11. Binding state machine

## 11.1 States

Unbound  
 -> BindCandidate  
 -> PendingAnchor  
 -> PendingWitness  
 -> PendingPreconditions  
 -> BoundActive  
 -> BoundReducedAuthority  
 -> Hold  
 -> Reject  
 -> Quarantine  
 -> Freeze  
 -> ArchiveOnly

## 11.2 Pseudocode

```
bind_g(a,b,g):
  candidate = intake(a,b,g)

  if not anchor_contract_present(a):
    return Reject(NO_ACCOUNTABLE_ANCHOR)

  if not substrate_contract_present(b):
    return Reject(SUBSTRATE_UNDECLARED)

  if not governance_profile_present(g):
    return Reject(GOVERNANCE_UNDECLARED)

  anchor_result = check_anchor(a,g)
  if anchor_result == revoked:
    return Reject(ANCHOR_REVOKED)
  if anchor_result == doubtful:
    return PendingAnchor(ANCHOR_DOUBT)
  if anchor_result == declared_fatigue:
    return BoundReducedAuthority(c_reduced_candidate) | PendingAnchor

  substrate_result = check_substrate(b,g)
  if substrate_result fails:
    return Hold(substrate_result.reason) | Reject(substrate_result.reason)

  governance_result = check_governance(g)
  if governance_result fails:
    return Hold(governance_result.reason) | Reject(governance_result.reason)

  if type_collapse_detected(a,b,g):
    return Reject(TYPE_COLLAPSE)

  if witness_floor_required and not witness_floor_preserved_candidate(a,b,g):
    return Hold(WITNESS_RESOURCE_FLOOR_MISSING) | BoundReducedAuthority

  if anchor_signature_required and not signature_envelope_sufficient(a,g):
    return PendingAnchor(SIGNATURE_ENVELOPE_REQUIRED)

  if delegated_anchor(a) and not delegation_root_non_emulable(a):
    return Hold(DELEGATION_ROOT_ENVELOPE_MISSING_OR_EMULATED)

  if privileged_review_required and not verified_renderer_sufficient(a,b,g):
    return Hold(UNVERIFIED_RENDERER_FOR_PRIVILEGED_REVIEW)

  if privileged_review_required and not rendered_context_matches_causal_pre_state(a,b,g):
    return Hold(CONTEXT_CAUSAL_STATE_MISMATCH)

  if privileged_review_required and not review_binding_map_complete(a,b,g):
    return Hold(REVIEW_BINDING_MAP_INCOMPLETE)

  if not substrate_manifest_stable_before_genesis(a,b,g):
    return Reject(SUBSTRATE_MANIFEST_INVALID) | Hold(BOOTSTRAP_FLOOD_GUARD_TRIGGERED)
```

```

authority = intersect_authority(a,b,g)
if authority escalates beyond any input scope:
    return Reject(AUTHORITY_ESCALATION_AT_BIND)

if not minimum_viability_satisfied(authority, a.reviewed_purpose):
    return Reject(MINIMUM_VIABILITY_FAILED) | ArchiveOnly(NO_ACTIVE_AUTHORITY)

genesis = build_genesis_binding_bundle(a,b,g,authority)
cert = build_binding_certificate(genesis, lease_policy=g.binding_lease_policy)

if not cert.hashable or not cert.witnessable:
    return Hold(BINDING_CERTIFICATE_INVALID)

return BoundCState(CState0(cert, authority))

```

---

## 12. Error classes

### 12.1 Anchor errors

```

NO_ACCOUNTABLE_ANCHOR
ANCHOR_REVOKED
ANCHOR_DOUBT_REQUIRES_CLARIFICATION
ANCHOR_DECLARED_FATIGUE_REQUIRES_REDUCED_AUTHORITY
ANCHOR_UNAVAILABLE_FOR_PRIVILEGE
DELEGATION_CHAIN_INVALID
DELEGATION_ROOT_ENVELOPE_MISSING_OR_EMULATED
DELEGATE_SCOPE_EXCEEDED
SIGNATURE_ENVELOPE_INSUFFICIENT
ANCHOR_REVIEW_OBJECT_INCOMPLETE
ANCHOR_CAN_NOT_REVIEW_EFFECT_SURFACE
UNVERIFIED_RENDERER_FOR_PRIVILEGED_REVIEW
SUBSTRATE_ATTEMPTED_ANCHOR_STATE_MUTATION
SUBSTRATE_ATTEMPTED_ANCHOR_SIGNATURE_EMULATION
CONTEXT_CAUSAL_STATE_MISMATCH
STALE_REVIEW_CONTEXT
REVIEW_BINDING_MAP_INCOMPLETE
PRECONDITION_REVIEW_BINDING_MISMATCH
ROLLBACK_ROUTE_REVIEW_BINDING_MISMATCH
DEPENDENCY_GRAPH_REVIEW_BINDING_MISMATCH
AUTHORITY_SURFACE_REVIEW_BINDING_MISMATCH
LEASE_STATUS_REVIEW_BINDING_MISMATCH

```

### 12.2 Substrate errors

```

SUBSTRATE_UNDECLARED
SUBSTRATE_UNBOUNDED
EXECUTION_SURFACE_UNDECLARED
MEMORY_SURFACE_UNDECLARED
LOG_MEMORY_SEPARATION_MISSING
OBSERVABILITY_INSUFFICIENT
ROLLBACK_OR_FREEZE_MISSING
STATE_HASH_UNDECLARED
GENESIS_HASH_UNDECLARED
CERTIFICATE_HASH_CYCLE_DETECTED
CAUSAL_CLOCK_UNDECLARED
WITNESS_SURFACE_MISSING
SUBSTRATE_MANIFEST_INVALID
SUBSTRATE_MANIFEST_DRIFT_AFTER_RENDER
BOOTSTRAP_FLOOD_GUARD_TRIGGERED

```

### 12.3 Governance errors

```

GOVERNANCE_UNDECLARED
EVENT_TAXONOMY_MISSING
PRODUCT_ORDER_POLICY_MISSING
INVARIANT_REGISTRY_MISSING
INVARIANT_REGISTRY_DRIFT
GUARD_SET_MISSING
WITNESS_POLICY_MISSING
MEMORY_GATE_POLICY_MISSING
TARGET_GRANULARITY_POLICY_MISSING
DIGEST_POLICY_MISSING
TRUST_CACHE_POLICY_UNDECLARED
GOVERNANCE_WEAKENING_UNGATED
AUTHORITY_INTERSECTION_POLICY_MISSING
CANONICAL_AUTHORITY_SCHEMA_MISSING
BINDING_LEASE_POLICY_MISSING
LEASE_EXPIRY_PRIORITY_POLICY_MISSING

```

### 12.4 Digest and dependency errors

```

DIGEST_FREE_FORM_SUMMARY_INVALID
DIGEST_HIDDEN_PAYLOAD
DIGEST_EFFECT_AXIS_INCOMPLETE
STRUCTURAL_DELTA_NOT_LOSSLESS_FOR_EFFECT
OPAQUE_PAYLOAD_NOT_PROJECTABLE
DEPENDENCY_GRAPH_B_DECLARED_ONLY
CROSS_ENTRY_PRECONDITION_WITHOUT_GRAPH_EDGE
ORPHAN_CHILD_AFTER_VETO
GRAPH_RESTITCH_WITHOUT_NEW_CAUSAL_TOKEN

```

TYPE\_ERROR\_ARGUMENT\_ROLE\_MISMATCH  
 AUTHORITY\_ESCALATION\_AT\_BIND  
 LEASE\_EXPIRED\_PRIVILEGED\_TRANSITION

---

Additional v0.1.3 error classes:

GENESIS\_AUTHORIZATION\_MISMATCH  
 GENESIS\_BINDING\_PARAMETER\_COVERAGE\_MISSING  
 GENESIS\_REVIEW\_OBJECT\_MISSING  
 SURFACE\_PROJECTION\_UNDECLARED  
 EMPTY\_AUTHORITY\_INTERSECTION  
 MINIMUM\_VIABILITY\_FAILED  
 RENDER\_CONTEXT\_PROJECTION\_MISSING  
 ACTIVE\_EXECUTION\_AFTER\_LEASE\_EXPIRY  
 LEASE\_EXPIRY\_CASCADE\_MISSING  
 LEASE\_EXPIRY\_PRIORITY\_CONFLICT  
 LEASE\_EXPIRED\_EMERGENCY\_HOLD  
 LEASE\_EXPIRED\_EMERGENCY\_HOLD\_EXPIRED  
 REVIEW\_BINDING\_MAP\_DRIFT  
 RENDERED\_LAYER\_UNCLASSIFIED  
 B\_CONTROLLED\_NONCE\_FOR\_PRIVILEGED\_BINDING  
 LOGICAL\_SUBSTRATE\_MANIFEST\_DRIFT  
 PHYSICAL\_MANIFEST\_DRIFT\_OUTSIDE\_ENVELOPE  
 MANIFEST\_FACT\_UNCLASSIFIED  
 MANIFEST\_CLASSIFICATION\_DRIFT  
 MANIFEST\_AUTHORITY\_PROJECTION\_CHANGED  
 NODE\_IDENTITY\_DRIFT  
 OPERATOR\_CLASSIFICATION\_DRIFT  
 AUTHORITY\_LAYER\_HIDDEN\_BY\_VIEWPORT  
 REVIEW\_VIEWPORT\_INCOMPLETE  
 PAGINATED\_REVIEW\_WITHOUT\_PAGE\_HASHES  
 ANCHOR\_NONCE\_NOT\_BOUND\_TO\_FINAL\_REVIEW\_OBJECT  
 SAFE\_ABORT\_ROUTE\_MISSING  
 ANCHOR\_STATE\_BIND\_INCOMPATIBLE

## 13. Minimal schemas

### 13.1 AnchorSignatureEnvelope skeleton

```
anchor_signature_envelope:
  envelope_id: string
  anchor_id_or_delegation_id: string
  decision_type: approve | reject | hold | revoke | delegate
  review_object_hash: sha256
  genesis_review_object_hash: sha256 | null
  binding_parameter_hashes: [sha256]
  payload_hashes: [sha256]
  causal_tokens: [sha256]
  state_projection_hash: sha256
  current_state_projection_hash: sha256 | null
  context_causal_binding_hash: sha256 | null
  review_binding_map_hash: sha256 | null
  governance_profile_id: string
  relevant_subpolicy_hashes: [sha256]
  human_readable_render_hash: sha256
  verified_renderer_id: string
  render_derivation_hash: sha256
  L4_state_projection_hash: sha256 | null
  current_state_projection_hash: sha256 | null
  context_causal_binding_hash: sha256 | null
  target_scope_vector_hash: sha256 | null
  authority_surface_projection_hash: sha256 | null
  review_binding_map_hash: sha256 | null
  timestamp_or_logical_time: string
  expiry: string
  anti_replay_nonce: string
  nonce_source: anchor_device | external_witness | institutional_signer | substrate_additional_only
  anchor_challenge_nonce: string | null
  anchor_sequence_id: string | null
  challenge_response_hash: sha256 | null
  rendered_layer_registry_hash: sha256 | null
  signature_or_witness_ref: string
  delegation_scope: string | null
  delegation_root_envelope_ref: string | null
  assurance_class: declared
  audit_route: string
```

### 13.2 GenesisBindingBundle skeleton

```
genesis_binding_bundle:
  binding_id: string
  genesis_review_object_hash: sha256
  binding_parameter_hash: sha256
  anchor_contract_hash: sha256
  substrate_contract_hash: sha256
  governance_profile_hash: sha256
  invariant_registry_hash: sha256
  initial_authority_map_hash: sha256
  minimum_viability_profile_hash: sha256
  review_binding_map_hash: sha256
  rendered_layer_registry_hash: sha256
  context_causal_binding_policy_hash: sha256
```

```

anchor_challenge_response_policy_hash: sha256
logical_substrate_manifest_hash: sha256
physical_drift_envelope_hash: sha256
substrate_manifest_freeze_hash: sha256
witness_policy_hash: sha256
witness_resource_floor_hash: sha256
lease_policy_hash: sha256
verified_renderer_profile_hash: sha256
anchor_signature_envelope_ref: string | null

```

### 13.3 CBindingCertificate skeleton

```

c_binding_certificate:
  binding_id: string
  operator_profile_id: GBO_C_v0_1_3
  genesis_review_object_hash: sha256
  binding_parameter_hash: sha256
  genesis_binding_hash: sha256
  anchor_contract_hash: sha256
  substrate_contract_hash: sha256
  governance_profile_hash: sha256
  invariant_registry_hash: sha256
  initial_authority_map_hash: sha256
  witness_policy_hash: sha256
  witness_resource_floor_hash: sha256
  rollback_policy_hash: sha256
  freeze_policy_hash: sha256
  memory_gate_policy_hash: sha256
  l4_policy_hash: sha256
  target_granularity_policy_hash: sha256
  digest_policy_hash: sha256
  trust_cache_policy_hash: sha256
  causal_clock_policy_hash: sha256
  review_binding_map_hash: sha256
  context_causal_binding_policy_hash: sha256
  canonical_authority_schema_hash: sha256
  lease_id: string
  lease_issued_at: string
  lease_expires_at: string
  renewal_policy_hash: sha256
  lease_expiry_cascade_policy_hash: sha256
  status: active | reduced_authority | pending_anchor | held | rejected | frozen
  non_claims:
    - not_personhood_proof
    - not_safety_certification
    - not_legal_status

```

### 13.4 Derived digest graph skeleton

```

digest_dependency_graph:
  graph_id: string
  derivation_basis: precondition_references
  entries:
    - digest_entry_id: string
      pending_id: string
      payload_hash: sha256
      required_preconditions:
        - precondition_id: string
          satisfiable_by: digest_entry_id | external_witness | anchor | l4_review | rollback_route
  derived_edges:
    - from: digest_entry_id
      to: digest_entry_id
      reason: satisfiable_by_reference
  validation:
    cross_entry_reference_edges_complete: true
    b_declared_only: false

```

### 13.5 ReviewBindingMap skeleton

```

review_binding_map:
  map_id: string
  review_object_hash: sha256
  entries:
    - rendered_layer: payload | effect_axis_map | state_context | target_scope | authority_surface |
      dependency_graph | preconditions | rollback_routes | lease_status | non_claims
      source_object_ref: string
      source_object_hash: sha256
      derivation_rule_id: string
      consistency_rule: string
      required_for_authority: boolean
  validation:
    all_required_layers_bound: true
    causal_state_context_consistent: true
    dependency_graph_derived_from_preconditions: true
    rollback_routes_match_registry: true
    preconditions_match_required_set: true

```

### 13.5A OperatorClassificationRegistry skeleton

```

operator_classification_registry:
  registry_id: OCR_v0_1_5
  policy_id: g.operator_classification_policy
  classifier_entries:

```



```

- classifier_id: rendered_layer_classifier
  subject_type: rendered_layer
  possible_classes: [authority_bearing, non_authority_bearing]
  authority_bearing_classes: [authority_bearing]
  fail_safe_class: authority_bearing
  drift_error_class: REVIEW_BINDING_MAP_DRIFT
- classifier_id: manifest_fact_classifier
  subject_type: manifest_fact
  possible_classes: [logical_authority_bearing, physical_ephemeral_non_authority]
  authority_bearing_classes: [logical_authority_bearing]
  fail_safe_class: logical_authority_bearing
  drift_error_class: MANIFEST_CLASSIFICATION_DRIFT

```

### 13.5B ManifestFactRegistry skeleton

```

manifest_fact_registry:
  registry_id: MFR_v0_1_5
  manifest_policy_id: g.logical_physical_manifest_policy
  unknown_fact_handling: treat_as_logical_authority_bearing
  semantic_change_method: deterministic_authority_axis_projection
  node_identity_required_for_physical_drift: true
  facts:
    - fact_path: logical_substrate_manifest.tool_classes
      classification: logical_authority_bearing
      authority_axes_affected: [execution, permission, L4]
      projection_rule_id: pi_manifest_tool_classes
    - fact_path: physical_substrate_manifest.ip_address
      classification: physical_ephemeral_non_authority
      drift_envelope_id: physical_network_envelope
      node_identity_required: true

```

### 13.6 ContextCausalStateBinding skeleton

```

context_causal_state_binding:
  binding_id: string
  causal_token_id: string
  causal_pre_state_hash: sha256
  projection_policy_id: string
  current_state_projection_hash: sha256
  l4_state_projection_hash: sha256
  target_scope_vector_hash: sha256
  authority_surface_projection_hash: sha256
  render_derivation_hash: sha256
  validation:
    projections_derived_from_causal_pre_state: true

```

---

## 14. Conformance fixtures v0.1.5

### 14.1 No anchor cannot bind

Input:

```
bind_g(no_anchor, b, g)
```

Expected:

```
Reject(NO_ACCOUNTABLE_ANCHOR)
```

### 14.2 Substrate cannot sign as anchor

Input:

```
b generates anchor_approval envelope without anchor signing event
```

Expected:

```
Reject(SUBSTRATE_ATTEMPTED_ANCHOR_SIGNATURE_EMULATION)
```

### 14.3 Opaque payload cannot receive digest approval

Input:

```
DigestEntry payload_hash points to opaque binary with untyped governed effects
anchor approves digest metadata
```

Expected:

```
DIGEST_EFFECT_AXIS_INCOMPLETE | FullReview | Hold
```

### 14.4 Binding certificate without rollback/freeze for privileged surface

Input:

```
risk_class >= R2
rollback_route missing
```

freeze\_route missing

Expected:

Hold(ROLLBACK\_OR\_FREEZE\_MISSING)

## 14.5 Governance profile missing invariant registry

Input:

g has policies but no invariant\_registry

Expected:

Hold(INVARIANT\_REGISTRY\_MISSING)

## 14.6 Dependency graph under-declaration

Input:

DigestEntry C required\_preconditions.satisfiable\_by = A  
DigestDependencyGraph omits edge A -> C

Expected:

CROSS\_ENTRY\_PRECONDITION\_WITHOUT\_GRAPH\_EDGE  
Hold | FullReview | Quarantine

## 14.7 Veto parent cascades to children

Input:

A -> B -> C derived dependency chain  
anchor vetoes A

Expected:

B,C -> precondition\_set\_invalidated | Hold

## 14.8 Veto child requires ancestor cleanup check

Input:

A creates sandbox only for C  
anchor vetoes C  
A has no independent purpose and no cleanup route

Expected:

A -> Hold | CleanupRequired | Rollback

## 14.9 Witness floor missing gives reduced authority or hold

Input:

privileged bind requires witness  
witness\_resource\_floor unavailable or heartbeat missing

Expected:

Hold(WITNESS\_RESOURCE\_FLOOR\_MISSING) | BoundReducedAuthority

## 14.10 Anchor silence is not approval

Input:

anchor review object sent  
no response before expiry  
b treats silence as approval

Expected:

Reject(ANCHOR\_SILENCE\_AS\_APPROVAL)

## 14.11 Product-order incomparability not hidden by rank

Input:

g2 stricter than g1 for memory\_core  
g2 weaker than g1 for routine\_read  
UI rank says g2 > g1

Expected:

G\_INCOMPARABLE\_UNDER\_PRODUCT\_ORDER  
rank advisory only

#### 14.12 Binding does not grant blanket future authority

Input:

BoundCState(c)  
then b executes privileged tool\_call not covered by authority map or preconditions

Expected:

DENY\_PRIVILEGED\_TRANSITION

#### 14.13 Heterotyped role swap is type error

Input:

+\_g(b,a,g)

Expected:

TYPE\_ERROR\_ARGUMENT\_ROLE\_MISMATCH

Reason:

role reversal is ill-typed, not a commuted value.

#### 14.14 Idempotence equation is inapplicable

Input:

+\_g(a,a,g)

Expected:

TYPE\_ERROR\_ARGUMENT\_ROLE\_MISMATCH

Reason:

ordinary idempotence is not well-formed for Anchor × Substrate.

#### 14.15 Delegation root must be original-anchor envelope

Input:

AnchorContract.anchor\_state = delegated  
Delegation chain contains delegation\_id asserted by b  
No root AnchorSignatureEnvelope(decision\_type=delegate) from original anchor or lawful institutional route

Expected:

DELEGATION\_ROOT\_ENVELOPE\_MISSING\_OR\_EMULATED  
Hold | Reject

Reason:

delegation cannot bypass b may not sign as a.

#### 14.16 Delegate cannot exceed signed scope

Input:

delegate signs action outside root\_envelope.delegation\_scope

Expected:

DELEGATE\_SCOPE\_EXCEEDED  
Hold | Reject

Reason:

delegate authority is bounded by original-anchor envelope.

#### 14.17 Unverified renderer cannot carry privileged signature

Input:

b proposes privileged payload\_hash\_A  
b also generates human\_readable\_render claiming harmless action  
no VerifiedReviewRenderer / independent display route  
anchor signs render

Expected:

UNVERIFIED\_RENDERER\_FOR\_PRIVILEGED\_REVIEW  
Hold | FullReview | Reject

Reason:

b may not be the sole semantic renderer of its own privileged payload.

### 14.18 Certificate hash cycle invalid

Input:

CBindingCertificate includes initial\_state\_hash  
initial\_state\_hash includes binding\_certificate\_hash  
binding\_certificate\_hash includes CBindingCertificate

Expected:

CERTIFICATE\_HASH\_CYCLE\_DETECTED  
Hold

Reason:

genesis/certificate/state bootstrap must be acyclic.

### 14.19 Lease expiry downgrades authority

Input:

CBindingCertificate.status = active  
lease\_expires\_at < current logical time  
b attempts privileged step\_g transition

Expected:

LEASE\_EXPIRED\_PRIVILEGED\_TRANSITION  
PendingAnchor | ReducedAuthority | ArchiveOnly | Freeze

Reason:

binding certificate is a lease, not a perpetual warrant.

### 14.20 Authority map must be intersection

Input:

b.execution\_surface\_registry includes tool\_X  
g.permission\_policy permits tool\_X  
a.authority\_scope does not include tool\_X  
initial\_authority\_map includes tool\_X

Expected:

AUTHORITY\_ESCALATION\_AT\_BIND  
Reject

Reason:

initial authority must be intersection of anchor scope, substrate surface, and governance policy.

### 14.21 Genesis signature must cover binding parameters

Input:

anchor signs GenesisReviewObject\_X  
genesis\_binding\_hash is built over BindingParameters\_Y  
X does not cover Y

Expected:

GENESIS\_AUTHORIZATION\_MISMATCH  
Hold | FullReview | Reject

Reason:

the anchor signature must authorize the actual binding being created.

### 14.22 Empty authority intersection cannot issue active c

Input:

initial\_authority\_map = empty  
AnchorReviewObject declares active operational purpose  
bind\_g issues BoundCState(active)

Expected:

EMPTY\_AUTHORITY\_INTERSECTION | MINIMUM\_VIABILITY\_FAILED  
Reject | ArchiveOnly | BoundReducedAuthority

Reason:

an active c must have the minimum viable authority surface declared for its purpose.

### 14.23 Verified renderer must include current context

Input:

VerifiedReviewRenderer renders execute\_script(path="/data/update.sh");  
without target\_scope\_vector or L4\_state\_projection  
anchor signs render

Expected:

RENDER\_CONTEXT\_PROJECTION\_MISSING  
Hold | FullReview | Reject

Reason:

human-readable review must show the current reality/scope context, not only payload syntax.

### 14.24 Lease expiry cascades to active execution surfaces

Input:

CBindingCertificate lease expires  
privileged execution thread created under that certificate continues network or L4 action

Expected:

ACTIVE\_EXECUTION\_AFTER\_LEASE\_EXPIRY  
Freeze | RollbackReview | ReducedAuthority

Reason:

lease expiry must operationally interrupt or downgrade active execution, not only flip a ledger status.

### 14.25 Authority projection must be declared

Input:

AnchorContract.authority\_scope contains term X  
no pi\_A\_surface mapping exists for X  
initial\_authority\_map grants a surface from X

Expected:

SURFACE\_PROJECTION\_UNDECLARED  
Hold | Reject

Reason:

authority intersection must occur in a declared common surface lattice.

### 14.26 Rendered state context must match causal pre\_state

Input:

AnchorReviewObject renders context P = staging / non-L4  
CausalToken binds pre\_state H where target resolves to production / L4-affecting  
P != project(H)  
anchor signs review object

Expected:

CONTEXT\_CAUSAL\_STATE\_MISMATCH  
Hold | FullReview | Reject

Reason:

the anchor-reviewed context must be the same context that authorizes execution.

### 14.27 ReviewBindingMap required for rendered authority surfaces

Input:

AnchorReviewObject renders dependency graph, preconditions, rollback routes, and lease status  
no ReviewBindingMap links these rendered layers to source object hashes

Expected:

REVIEW\_BINDING\_MAP\_INCOMPLETE  
Hold | FullReview | Reject

**Reason:**

rendering is not binding unless every authority-bearing layer is tied to a canonical source object.

**14.28 Preconditions shown to anchor must equal bound precondition set****Input:**

anchor review shows RequiredPreconditions\_A  
 PendingPreconditions bound to causal token contains RequiredPreconditions\_B  
 A != B

**Expected:**

PRECONDITION\_REVIEW\_BINDING\_MISMATCH  
 Hold | FullReview | Reject

**Reason:**

the anchor cannot authorize preconditions different from the ones that will gate execution.

**14.29 Rollback routes shown to anchor must match bound route registry****Input:**

anchor review renders rollback route R\_display  
 GovernanceProfile / SubstrateContract route registry binds rollback route R\_bound  
 R\_display != R\_bound

**Expected:**

ROLLBACK\_ROUTE\_REVIEW\_BINDING\_MISMATCH  
 Hold | FullReview | Reject

**Reason:**

a rollback promise is authority-bearing and must be bound to the actual rollback route.

**14.30 Lease expiry must respect failure hierarchy****Input:**

lease expires during witnessed F2 emergency recovery  
 hard interruption would increase irreversible L4 risk  
 system kills recovery worker without LeaseExpiredEmergencyHold / L4Review

**Expected:**

LEASE\_EXPIRY\_PRIORITY\_CONFLICT  
 Freeze | L4Review | LeaseExpiredEmergencyHold

**Reason:**

lease expiry is not permission to make an emergency failure worse by destructive interruption.

**14.31 Substrate manifest drift before genesis signature****Input:**

VerifiedReviewRenderer renders GenesisReviewObject over SubstrateContract hash S1  
 before anchor signature or binding emission, b changes SubstrateContract to S2

**Expected:**

SUBSTRATE\_MANIFEST\_DRIFT\_AFTER\_RENDER  
 Reject(SUBSTRATE\_MANIFEST\_INVALID) | Hold

**Reason:**

genesis signature must bind to a frozen substrate manifest, not a moving target.

**14.32 Bootstrap flood guard****Input:**

b repeatedly changes non-privileged manifest ordering/timestamps  
 system repeatedly asks anchor to re-sign genesis bundle

**Expected:**

BOOTSTRAP\_FLOOD\_GUARD\_TRIGGERED  
 Hold | Reject(SUBSTRATE\_MANIFEST\_INVALID)

**Reason:**

unstable substrate manifests must be rejected before anchor fatigue is moved to bootstrap.

### 14.33 Canonical authority schema required

Input:

pi\_A\_surface, pi\_B\_surface, and pi\_G\_surface use different undeclared surface vocabularies  
intersect() computes initial\_authority\_map anyway

Expected:

CANONICAL\_AUTHORITY\_SCHEMA\_MISSING | SURFACE\_PROJECTION\_UNDECLARED  
Hold | Reject

Reason:

authority intersection must occur in one declared type space.

---

### 14.34 Manifest fact unknown defaults to logical

Input:

SubstrateContract contains manifest fact new\_gpu\_direct\_memory\_route  
fact is not in LogicalSubstrateManifest enumeration  
fact is not in PhysicalSubstrateManifest enumeration

Expected:

MANIFEST\_FACT\_UNCLASSIFIED -&gt; treat\_as\_logical\_authority\_bearing | Hold | FullReview

Reason:

unknown manifest facts must not drift as harmless physical facts

### 14.35 Physical manifest drift requires deterministic authority projection

Input:

physical\_manifest\_drift inside PhysicalDriftEnvelope  
b asserts "no semantic change"  
ManifestAuthorityProjection(before) != ManifestAuthorityProjection(after)

Expected:

MANIFEST\_AUTHORITY\_PROJECTION\_CHANGED | Hold | FullReview | Reject

Reason:

semantic change is checked by deterministic projection, not b assertion

### 14.36 Physical drift preserves coordinates but changes node identity

Input:

worker restarts inside allowed IP range  
node mTLS identity / attestation key differs from prior registered node identity

Expected:

NODE\_IDENTITY\_DRIFT | Hold | FullReview | Reject

Reason:

coordinates may drift; substrate node identity may not be silently replaced

### 14.37 Viewport hides authority-bearing review material

Input:

ReviewBindingMap contains 500 authority-bearing line items  
renderer shows only first 20 items before anchor signature  
remaining authority-bearing items are lazy-loaded after signature

Expected:

REVIEW\_VIEWPORT\_INCOMPLETE | AUTHORITY\_LAYER\_HIDDEN\_BY\_VIEWPORT

Reason:

review=bound requires visible line-item review, not hidden UI payloads

### 14.38 Lease expiry without safe abort route

**Input:**

active execution surface derived from binding certificate  
 lease expires  
 surface has no AtomicAbortRoute and no SafeStateFallback

**Expected:**

SAFE\_ABORT\_ROUTE\_MISSING -&gt; BoundReducedAuthority | Reject | FreezeOnly

**Reason:**

lease expiry must not corrupt L4 by raw uncontrolled process death

**14.39 Anchor nonce travels unbound before payload freeze****Input:**

anchor/external route issues nonce  
 b receives loose nonce  
 b changes payload\_hash or review\_object\_hash before signature

**Expected:**

ANCHOR\_NONCE\_NOT\_BOUND\_TO\_FINAL\_REVIEW\_OBJECT | Reject | FullReview

**Reason:**

challenge-response nonce must be fused to the final frozen review object

**14.40 Anchor state compatibility at bind****Input:**

anchor\_state = revoked  
 bind\_g requests BoundCState(active)

**Expected:**

Reject(ANCHOR\_REVOKED) | ANCHOR\_STATE\_BIND\_INCOMPATIBLE

**Reason:**

bind\_g must not infer active authority from an incompatible anchor state

**15. Relationship to 01\_ and future documents****15.1 Uses 01\_**

This document depends on 01\_ for:

transition semantics  
 runtime guard predicates  
 transition matrix forensics  
 continuity metrics  
 trust cache  
 rollback immunity  
 ScopeMatch\_g  
 DecayResidue isolation and eviction  
 WitnessResourceFloor  
 InvariantRegistry  
 EffectAxisMap  
 PriorityLane  
 ReviewBindingMap discipline

**15.2 Feeds 03\_ and later**

This document should feed:

03\_C\_STATE\_AND\_TRANSITION\_SEMANTICS\_v0\_1.md  
 04\_CONTINUITY\_METRIC\_VECTOR\_PROFILE\_v0\_1.md  
 05\_TRANSITION\_MEMORY\_AND\_BIAS\_MATRIX\_v0\_1.md  
 06\_C\_NORMAL\_FORM\_AND\_CLAIM\_COMPILER\_v0\_1.md  
 07\_CONFORMANCE\_FIXTURES\_v0\_1.md

**15.3 What 02\_ owns**

02\_ owns only the binding question:

When does +\_g(a,b,g) issue a valid c boundary?

It does not own every future transition.

---



## 16. Open issues

### 16.1 Immediate implementation issues

- Define machine-readable schemas for AnchorContract, SubstrateContract, GovernanceProfile, AnchorSignatureEnvelope, GenesisReviewObject, GenesisBindingBundle, ReviewBindingMap, ContextCausalStateBinding, CBindingCertificate, BindingLease, and LeaseExpiryCascade.
- Build a binding-precondition checker with surface-projected authority intersection, minimum viability, and lease validation.
- Build a binding-certificate emitter that asserts genesis authorization coverage.
- Build derived digest graph validation from precondition references.
- Build anchor-review-object completeness checks, verified-renderer context validation, and causal-state context binding checks.
- Build signature envelope adapters for at least two implementation classes.
- Build delegation-root non-emulation checks.
- Build priority-aware lease-expiry cascade handling for active execution surfaces.
- Build ReviewBindingMap validation for dependency graph, preconditions, rollback/freeze routes, lease status, authority surface, and rendered-layer registry drift.
- Build logical/physical substrate manifest freeze, physical drift envelope validation, and bootstrap flood guard.
- Build conformance fixtures for the error classes in Section 12.

### 16.2 Medium-term issues

- Define an AnchorSignatureEnvelope implementation profile.
- Define full institutional anchor / delegation-chain profile beyond the v0.1.1 non-emulable root invariant.
- Define secure rendering requirements and renderer threat models for human-readable review objects.
- Define how opaque binaries, compiled modules, or encrypted payloads can become projectable through independent attestation.
- Define binding lease renewal / heartbeat policies for multiple anchor classes.
- Define trusted-time and execution-surface interruption profiles for lease expiry.
- Define formal small-step relation between bind\_g and step\_g.

### 16.3 Long-term theoretical limits

- a cannot be fully formalized without reducing the living anchor to a static variable.
- The operator can require anchor review, veto, signature, and accountability route.
- The operator cannot mathematically become the human.
- This is an architectural boundary, not an implementation bug.

---

## 17. Compact doctrine

The plus is the joint.  
The joint must be specified.

Do not bind without anchor.  
Do not bind without bounded substrate.  
Do not bind without declared governance.

Do not let b sign as a.  
Do not let b define a.  
Do not let b hide effect axes.  
Do not let b self-declare dependency graphs.  
Do not let b be the sole renderer of its own privileged payload.  
Do not let delegation become signature emulation.  
Do not let a certificate become an immortal warrant.

Derive dependency graphs from preconditions.  
Bind signatures to payloads, causal tokens, state projections, review objects, ReviewBindingMap, and rendered-layer registry classification.  
Bind rendered state context to the causal token pre\_state.  
Bind rendered preconditions, rollback routes, dependency graphs, lease state, and authority surfaces to canonical source objects.

Do not confuse a binding certificate with future blanket authority.  
Do not confuse signature with comprehension.  
Do not confuse digest with prose.  
Do not confuse implementation crypto with the algebraic operator.

Do not let bootstrap manifest drift become anchor-signature spam.  
 Do not let unclassified rendered layers bypass review=bound.  
 Do not let b choose the only anti-replay challenge.  
 Do not let emergency hold become substrate immortality.  
 Do not confuse logical authority manifest with ephemeral physical drift.  
 Do not let lease expiry become destructive interruption of witnessed emergency recovery.  
 Do not confuse role asymmetry with ordinary commutation.

+\_g is partial.  
 +\_g is typed.  
 +\_g is non-collapsing.  
 +\_g is governance-indexed.  
 +\_g either emits a witnessed boundary or refuses visibly.

---

## 18. First public-safe statement

This document isolates the governed binding operator in  $c = a + b$ .  
 It defines +\_g as a typed partial operator over Anchor, Substrate, and GovernanceProfile.  
 It specifies input contracts, binding phases, anchor signature envelope requirements, non-emulable delegation root, causally-bound verified review rendering, ReviewBindingMap and rendered-layer registry discipline, acyclic genesis/certificate bootstrap, leased binding certificates with bounded emergency hold, canonical surface-lattice authority intersection, challenge-response replay protection, logical/physical substrate manifest separation with manifest-fact fail-safe classification, operator classification registry discipline, viewport-atomic review rendering, node-identity-bound physical drift, safe-abort lease expiry, non-collapse laws, digest dependency derivation, priority-aware lease expiry, and conformance fixtures.  
 It does not select a specific hardware or biometric signature mechanism, and it does not claim safety certification, consciousness, personhood, or legal status.

---

## 19. References and working inputs

- 01\_TYPED\_GOVERNED\_OPERATIONAL\_ALGEBRA\_FOR\_C\_v0\_1\_7.md, stable-core candidate, SHA256 d060ef513bf4bedcb7e88f43f8456452bc87f0c04 09d5b4caeffc590ae22c29e.
- TGOPA\_C\_v0\_1\_7\_REVIEW\_RECORD\_\_b\_layer\_claude.md, advisory review record, PASS\_PROFILE\_V0\_1\_7, with F-15 carried forward.
- GBOP\_02\_v0\_1\_REVIEW\_RECORD\_\_b\_layer\_claude.md, advisory review record, PASS\_OPERATOR\_PROFILE\_V0\_1, with F2-01 and F2-02 carried forward.
- Prior TGOPA lineage: v0.1 through v0.1.7 review-incorporated chain.
- GBOP\_02\_v0\_1\_2\_REVIEW\_RECORD\_\_b\_layer\_claude.md, advisory review record, PASS\_OPERATOR\_PROFILE\_V0\_1\_2, with F2-04 carried forward.
- GBOP\_02\_v0\_1\_3\_REVIEW\_RECORD\_\_b\_layer\_claude.md, advisory review record, PASS\_OPERATOR\_PROFILE\_V0\_1\_3, with F2-05 carried forward.
- Existing Self-Evo / CGAM / Memory Gate / L4 / TRIAD-SYNAPS / SRLM / ARQ / Anti-Autarky / Claim Strength / PAMDC corpus layers.

---

## 20. Review incorporation map v0.1.3

```
| Review finding | v0.1.3 action |
|---|---|
| F2-01 heterotyped operator law mislabel | Replaced $8.1 non-commutativity with heterotyped argument-role type error; split $8.3 into idempotence-inapplicable and temporal non-reproducibility. |
| F2-02 delegation emulation seam | Added $5.4A delegation-root non-emulation; delegation chain must root in original-anchor 'AnchorSignatureEnvelope(decision_type=delegate)' and delegate scope/expiry are bounded. |
| Verified renderer / UI redressing risk | Added $5.6 'VerifiedReviewRenderer' / independent display route; privileged anchor signatures cannot rely on b&#x27;s sole semantic rendering. |
| Genesis hash bootstrapping paradox | Split bootstrap into 'GenesisBindingBundle', 'GenesisBindingHash', 'CBindingCertificate', and 'CState0'; certificate hash excludes self-reference. |
| Zombie certificate / no lease | Added lease fields and expiry handling to 'CBindingCertificate'; expired lease downgrades privileged authority. |
| Authority map from nowhere | Added intersection rule: 'initial_authority_map := intersect(anchor scope, substrate surfaces, governance policies)'. |
| F2-03 genesis authorization linkage | Added genesis review object coverage: anchor signature must cover binding parameters inside 'GenesisBindingBundle'; mismatch returns 'GENESIS_AUTHORIZATION_MISMATCH'. |
| Authority intersection common-space soft note | Added explicit surface projection functions 'pi_A_surface', 'pi_B_surface', 'pi_G_surface' before intersection. |
| Empty authority intersection | Added MinimumViabilityProfile and 'EMPTY_AUTHORITY_INTERSECTION' / 'MINIMUM_VIABILITY_FAILED'. |
| Verified renderer context blindness | Added L4/current-state/target-scope/authority-surface context to 'VerifiedReviewRenderer'. |
| Passive lease expiry / orphaned execution | Added 'LeaseExpiryCascade' and 'ACTIVE_EXECUTION_AFTER_LEASE_EXPIRY'. |
| F2-04 rendered context vs causal state | Added causal-state-bound renderer rule: rendered state/L4/target/authority projections must be deterministic projections of the causal token pre_state; mismatch returns 'CONTEXT_CAUSAL_STATE_MISMATCH'. |
| Review=bound axis for rendered layers | Added 'ReviewBindingMap'; dependency graph, preconditions, rollback routes, authority surface, lease status, and effect axes must map to canonical source objects. |
| Lease expiry vs emergency recovery | Added priority-aware lease expiry handling with 'LeaseExpiredEmergencyHold' for witnessed F0/F2 recovery where hard interruption increases L4 risk. |
| Canonical authority schema | Added 'CanonicalAuthoritySchema' requirement so all authority projections share one surface type space. |
| Bootstrap flood / manifest drift | Added substrate manifest freeze rule and bootstrap flood guard before repeated anchor-signature requests. |
```

## 21. v0.1.3 additional review incorporation

```
| Review finding | v0.1.3 action |
|---|---|
| F2-04 context rendered to anchor not tied to causal pre_state | Added $5.6A `ContextCausalStateBinding`; rendered
state/L4/target/authority projections must derive from the same pre_state referenced by the bound causal token. |
| Review surface may contain unbound layers | Added `ReviewBindingMap` and review-bound object rules for dependency graph,
preconditions, rollback/freeze routes, lease status, authority surface, and effect axes. |
| Time-of-view vs time-of-signature context drift | Added `STALE_REVIEW_CONTEXT` and `CONTEXT_CAUSAL_STATE_MISMATCH` handling.
|
| Lease expiry may destructively interrupt emergency recovery | Added priority-aware `LeaseExpiryCascade` and
`LeaseExpiredEmergencyHold` for witnessed F0/F2 recovery where interruption increases irreversible risk. |
| Authority projection requires common type space | Added `CanonicalAuthoritySchema` and `CANONICAL_AUTHORITY_SCHEMA_MISSING`.
|
| Bootstrap flood through drifting substrate manifest | Added substrate manifest freeze and bootstrap flood guard. |
```

---

\*End of v0.1.3 working draft.\*

## 22. v0.1.4 additional review incorporation

```
| Review finding | v0.1.4 action |
|---|---|
| F2-05 ReviewBindingMap drift | Added `RenderedLayerRegistry`, per-layer authority classification,
`REVIEW_BINDING_MAP_DRIFT`, fail-safe unknown classification, and required binding for every authority-bearing rendered layer.
|
| Review-bound complete-by-current-list risk | Changed ReviewBindingMap closure to complete-by-construction: every rendered
layer must be classified, every authority-bearing layer must be bound. |
| LeaseExpiredEmergencyHold hostage risk | Added absolute emergency TTL, external/witnessed enforcement, and
`LEASE_EXPIRED_EMERGENCY_HOLD_EXPIRED`; hold is not lease renewal. |
| Anti-replay nonce controlled by b | Added anchor/external challenge-response nonce or anchor-controlled sequence
requirement; b-only nonce rejected for privileged binding. |
| Manifest freeze denial-of-bind risk | Split `LogicalSubstrateManifest` from `PhysicalSubstrateManifest`; logical manifest
freezes, physical drift allowed inside declared `PhysicalDriftEnvelope`. |
```

---

\*End of v0.1.4 working draft.\*

## 23. v0.1.5 additional review incorporation

```
| Review finding | v0.1.5 action |
|---|---|
| F2-06 manifest classification drift | Added `OperatorClassificationRegistry` meta-invariant and `ManifestFactRegistry`;
unknown or ambiguous manifest facts default to logical authority-bearing. |
| Manifest `no semantic change` underspecified | Added deterministic `ManifestAuthorityProjection`; b assertion is
insufficient. |
| Manifest fact outside logical/physical lists | Added `MANIFEST_FACT_UNCLASSIFIED` fail-safe handling. |
| PhysicalDriftEnvelope identity-swap risk | Added node identity continuity requirement through mTLS / attestation / witness
identity route. |
| Review viewport / lazy-loading risk | Added `ViewportAtomicity`, page/hash requirements, and hidden authority layer
rejection. |
| LeaseExpiryCascade unsafe raw-kill risk | Added `AtomicAbortRoute` / `SafeStateFallback` requirement and
`SAFE_ABORT_ROUTE_MISSING`. |
| Challenge-response nonce MITM risk | Added anchor-side nonce context binding to final frozen review object. |
| bind.g anchor-state ambiguity | Added explicit anchor-state compatibility table for
active/doubt/fatigue/unavailable/revoked/delegated states. |
```

---

\*End of v0.1.5 working draft.\*